

UNIFESO

Centro Universitário Serra dos Órgãos

Curso de Análise e Desenvolvimento de Sistemas

MVP System

Documentação Oficial

Portal de Gestão de Projetos ADS

Equipe de Desenvolvimento

Amanda Lisbôa · Jhonathan Pegoral

Orientação: Prof. Rodrigo Braga

Teresópolis/Guapimirim — RJ | 2026

Sumário

Sumário	2
1. Objetivo e Finalidade da Solução — Visão Geral do Projeto	3
2. Funcionalidades Desejadas	5
3. Requisitos Funcionais e Não Funcionais	8
3.1 Requisitos Funcionais — Backend	8
3.2 Requisitos Não Funcionais — Backend	11
3.3 Requisitos Funcionais — Frontend	13
3.4 Requisitos Não Funcionais — Frontend	16
4. Desenvolvimento da Solução	19
4.1 Arquitetura da Aplicação	19
4.2 Modelagem de Dados	19
4.3 Diagramas de Casos de Uso	21
4.4 Diagramas de Sequência	26
5. Execução do Backend	30
5.1 Estrutura de Packages	30
5.2 Endpoints da API RESTful	34
5.3 Respostas HTTP	36
6. Execução da Interface — Frontend	39
7. Tecnologias Utilizadas	51
7.1 Backend	51
7.2 Frontend	52
7.3 Banco de Dados	53
7.4 Infraestrutura e DevOps	53

1. Objetivo e Finalidade da Solução — Visão Geral do Projeto

O MVP System é uma solução corporativa Web concebida estrategicamente para o gerenciamento de projetos acadêmicos no curso de Análise e Desenvolvimento de Sistemas (ADS) do UNIFESO (Centro Universitário Serra dos Órgãos). O sistema foi planejado para mitigar um desafio crítico no ecossistema educacional: a descentralização e a complexidade no acompanhamento do ciclo de vida dos projetos práticos (MVPs) desenvolvidos pelos discentes ao longo da graduação.

O gargalo central solucionado pela plataforma é a fragmentação crônica das informações acadêmicas — dados consolidados de alunos, formação de equipes de trabalho, temas e escopos de projetos atribuídos, cronogramas e status de andamento, agendamento de apresentações, lançamento de notas e centralização de repositórios. Sem uma ferramenta dedicada, esse fluxo de dados ficava sujeito a falhas por depender de planilhas locais, e-mails isolados e registros manuais.

O MVP System atua como um painel de controle centralizado de uso exclusivo do corpo docente e da coordenação (Professores/Administradores), não prevendo a interação ou o acesso direto por parte dos alunos. Dessa forma, todas as ações operacionais — desde o cadastro base de estudantes e a montagem de equipes até a avaliação individual final e a exportação de relatórios consolidados — são executadas de maneira segura pelo professor devidamente autenticado.

O escopo do sistema integra as seguintes entidades e conceitos acadêmicos:

Entidade	Descrição
Alunos	Registros mantidos pelo corpo docente contendo dados cadastrais: matrícula institucional, nome completo e contatos. Os alunos são associados diretamente a grupos específicos.
Grupos (Equipes)	Unidades organizacionais que reúnem alunos focados em uma linha tecnológica específica (Frontend, Backend ou Mobile), contendo período letivo vigente, data de criação, status operacional e link do repositório de código.

Projetos	Escopos técnicos e temas definidos pelo corpo docente que podem ser vinculados às equipes, detalhando a descrição do problema, requisitos mínimos, categoria tecnológica e o ano letivo correspondente.
Entregas	Histórico de apresentações das equipes, registrando datas das apresentações, pareceres gerais do avaliador e as notas individuais de cada aluno.
Professores Administradores	/ Usuários do sistema com autonomia total para gerenciar todas as entidades, cadastrar novos docentes e gerenciar credenciais de acesso.

2. Funcionalidades Desejadas

A solução contempla os seguintes módulos e macro-funcionalidades mapeados para o atendimento das regras de negócio:

Módulo de Controle de Acesso e Segurança

- Autenticação segura de docentes por meio de credenciais exclusivas (usuário/e-mail e senha) com geração de tokens temporários.
- Encerramento seguro de sessão com invalidação imediata dos tokens de acesso para prevenção de acessos indevidos.
- Mecanismo automático de renovação de sessão para garantir a continuidade do trabalho do docente sem interrupções.
- Proteção nativa de telas, impedindo que usuários não autenticados acessem o painel interno.
- Gerenciamento de contas de novos professores, permitindo a criação de novos usuários administrativos e a atualização de senhas.
- Listagem geral do corpo docente cadastrado e capacidade de desativação lógica de contas antigas.

Módulo de Gestão de Alunos

- Cadastro abrangente de alunos capturando matrícula, nome completo, e-mail institucional e celular.
- Manutenção de dados cadastrais (atualizações totais ou parciais).
- Exclusão de registros com trava de segurança: o sistema impede a remoção de alunos vinculados a equipes ativas.
- Painel de busca avançada com filtros por nome, número de matrícula ou e-mail.
- Ficha detalhada do estudante contendo seu histórico de grupos, notas e projetos associados.
- Mecanismo de alocação e remoção de alunos em suas respectivas equipes de projeto.

Módulo de Gestão de Grupos (Equipes)

- Abertura de novas equipes especificando nome, período acadêmico, tipo de MVP focado e link do repositório de desenvolvimento.
- Geração inteligente e automatizada de códigos de identificação para cada equipe com base em sua categoria.
- Associação em lote de múltiplos alunos a uma equipe no ato do cadastro ou em revisões posteriores.
- Trava de segurança para exclusão: impedimento de exclusão de equipes que contenham membros alocados ou projetos associados.
- Painel de filtragem avançada por ano letivo, período letivo, termos textuais ou equipes sem nenhum projeto vinculado.
- Atualização rápida de status da equipe diretamente na listagem visual.

Módulo de Gestão de Projetos

- Cadastro de temas de projetos com título, descrição do escopo, requisitos tecnológicos exigidos e status de atividade.
- Edição de termos e diretrizes de avaliação de projetos existentes.
- Painel de controle com filtros por tipo de MVP e ano corrente.
- Associação dirigida de projetos a grupos de trabalho.

Módulo de Entregas e Avaliações

- Listagem gerencial de equipes elegíveis para avaliação com ferramentas de pesquisa avançada.
- Persistência automática de filtros de busca para evitar a perda de contexto durante a navegação do professor.
- Painel de lançamento de avaliações individuais contemplando data da apresentação, parecer institucional e notas em escala regulamentada (0–6).

- Alteração automática de status da equipe para “Concluído” assim que todas as avaliações dos integrantes forem consolidadas.

Módulo de Indicadores Estratégicos (Dashboard)

- Resumo numérico contendo o total de projetos cadastrados, equipes ativas e entregas concluídas.
- Filtros dinâmicos globais por ano letivo e ciclo acadêmico.
- Gráficos estatísticos demonstrando taxas de conclusão por tecnologia, distribuição volumétrica de equipes e médias gerais de notas.
- Exportação de dados consolidados em formato de planilha eletrônica (.xlsx) para auditoria ou lançamento em sistemas externos.

Módulo de Perfil e Administração Geral

- Exibição dos dados do usuário atualmente conectado.
- Atualização interna de senhas corporativas.
- Painel para convite e cadastro de novos professores parceiros.

3. Requisitos Funcionais e Não Funcionais

Esta seção apresenta os requisitos que guiam o desenvolvimento do MVP System, organizados por camada (Backend e Frontend) e por natureza (funcionais e não funcionais), no formato de User Stories.

3.1 Requisitos Funcionais — Backend

RF-01: Autenticação de Usuário e Gestão de Sessão

O sistema deve permitir que um professor se autentique informando suas credenciais de acesso exclusivas; validar os dados contra a base de usuários e retornar tokens de acesso seguros em caso de sucesso; recusar o acesso e retornar sinalização de erro padronizada para credenciais inválidas; disponibilizar rotina de renovação automatizada do tempo de sessão via tokens de atualização; e permitir o encerramento voluntário da sessão, invalidando os tokens ativos.

User Stories:

- Como professor, quero realizar o login com meu usuário e senha para acessar as áreas administrativas restritas do sistema.
- Como professor, quero que minha sessão seja mantida ativa e segura enquanto utilizo a plataforma para não precisar refazer o login constantemente.
- Como professor, quero encerrar minha sessão ao finalizar meu trabalho para garantir que nenhuma outra pessoa acesse meus dados.

RF-02: Gestão de Administradores e Controle de Contas

O sistema deve permitir que um professor autenticado cadastre novos perfis de administradores/professores; validar a unicidade do nome de usuário e do e-mail cadastrado, impedindo duplicidades; exigir senha com padrões mínimos de segurança; permitir consulta do perfil do usuário logado e listagem dos professores cadastrados; e permitir a desativação lógica (soft delete) de uma conta de professor.

User Stories:

- Como administrador, quero cadastrar novos professores na plataforma para que eles gerenciem seus próprios grupos de alunos.
- Como administrador, quero visualizar a listagem de todos os professores cadastrados para monitorar quem possui acesso à ferramenta.
- Como administrador, quero desativar a conta de um docente desligado da instituição para revogar suas permissões imediatamente.

RF-03: Alteração de Credenciais de Acesso

O sistema deve permitir que o professor atualmente logado modifique sua própria senha de acesso; validar se a senha atual informada está correta antes de efetuar a troca; e impedir que a nova senha seja idêntica à senha utilizada anteriormente.

User Stories:

- Como professor, quero alterar minha senha corporativa periodicamente para manter a segurança da minha conta atualizada.

RF-04: Manutenção e Cadastro de Alunos

O sistema deve permitir o cadastro de alunos armazenando nome completo, matrícula institucional, e-mail e celular; garantir a obrigatoriedade e unicidade estrita da matrícula e do e-mail acadêmico; restringir a matrícula ao padrão regulamentado (máximo de 8 caracteres); permitir alteração total ou parcial dos dados cadastrais; e retornar a listagem de todos os alunos em ordem alfabética por padrão.

User Stories:

- Como professor, quero cadastrar os alunos do curso de ADS fornecendo seus dados institucionais para que fiquem disponíveis no sistema.
- Como professor, quero atualizar os dados de contato de um aluno para corrigir eventuais erros de digitação.
- Como professor, quero visualizar a ficha de um estudante específico para checar suas informações cadastrais básicas.

RF-05: Exclusão Controlada de Alunos

O sistema deve permitir a remoção definitiva de um aluno da base de dados e validar se o aluno possui vínculo com equipes de projeto acadêmico, bloqueando a exclusão caso o vínculo exista.

User Stories:

- Como professor, quero deletar um aluno que trancou o curso, desde que ele não faça parte de nenhum grupo ativo de MVP.

RF-06: Vinculação de Alunos a Equipes e Lançamento de Avaliações

O sistema deve permitir a vinculação de um aluno a um grupo de trabalho; permitir a remoção do vínculo; permitir o lançamento e a atualização da nota individual do aluno; validar se a nota está na escala de 0 a 6; e exigir que o aluno esteja previamente vinculado à equipe para permitir o lançamento da nota.

User Stories:

- Como professor, quero alocar os alunos em suas respectivas equipes de trabalho para organizar os grupos da disciplina.
- Como professor, quero lançar a nota de um aluno diretamente no painel do seu grupo para computar sua avaliação prática.

RF-07: Gerenciamento de Equipes (Grupos de MVP)

O sistema deve permitir o cadastro de novos grupos de trabalho especificando nome, data de criação, ciclo/período letivo, link de repositório e tipo de MVP; impor unicidade do nome do grupo; permitir alteração total ou parcial dos dados; e fornecer a listagem de todas as equipes por padrão a partir da data de criação mais recente.

User Stories:

- Como professor, quero cadastrar as equipes de MVP definindo a linha tecnológica que seguirão para dar início ao acompanhamento do período.
- Como professor, quero alterar o link do repositório de uma equipe caso eles atualizem o local do código do projeto.

RF-08: Exclusão Controlada de Equipes

O sistema deve permitir a remoção de um grupo de trabalho da base de dados; bloquear a exclusão caso a equipe possua alunos vinculados; e bloquear a exclusão caso a equipe possua um projeto associado com status Ativo.

User Stories:

- Como professor, quero deletar uma equipe criada por engano, garantindo que ela não tenha membros ou projetos ativos associados para não gerar inconsistências.

RF-09: Modelagem Base de Projetos e Acompanhamento de Entregas

O sistema deve manter uma estrutura de dados para armazenar informações de projetos (título, descrição, requisitos de avaliação, ano e status de atividade) e uma estrutura para registrar os cronogramas de entregas (descrição, data da apresentação, link de apoio e indicadores de realização da banca de avaliação).

User Stories:

- Como analista de sistemas, preciso que o núcleo de dados da aplicação possua os modelos estruturais de projetos e entregas definidos para dar suporte às expansões de funcionalidades das interfaces.

3.2 Requisitos Não Funcionais — Backend

RNF-01: Tecnologia e Linguagem de Programação

O sistema backend deve ser integralmente desenvolvido utilizando a linguagem Python e o framework Django, adotando a extensão Django REST Framework para garantir uma arquitetura de API padronizada, performática e de fácil manutenção.

RNF-02: Mecanismo de Segurança e Proteção de Dados

A camada de serviços deve utilizar tokens JWT para o controle de todas as requisições privadas. Os tokens de acesso devem possuir ciclo de vida curto (15

minutos) e os tokens de renovação devem possuir validade de 7 dias, com políticas estritas de rotatividade e inclusão em listas de desativação (blacklist) após o uso ou encerramento de sessão. Todas as credenciais devem ser armazenadas utilizando criptografia de hash seguro no banco de dados.

RNF-03: Camada de Persistência e Banco de Dados

O armazenamento relacional deve ser gerenciado pelo MySQL (versão 8.0). O mapeamento de tabelas deve impor integridade referencial nativa e o isolamento completo de credenciais de acesso por meio de variáveis de ambiente seguras.

RNF-04: Arquitetura de Software e Padrões de Projeto

A aplicação deve seguir o padrão de arquitetura em camadas, isolando a responsabilidade de roteamento e recepção (Views), transformação e validação de dados (Serializers) e centralização de regras de negócio complexas por meio de uma camada exclusiva de serviços (Service Layer).

RNF-05: Padronização de Respostas da API

Todas as respostas fornecidas pelo servidor devem adotar uma estrutura de dados uniforme contendo os objetos de retorno, mensagens informativas em português e o respectivo código de status HTTP.

RNF-06: Tratamento Global de Falhas

O sistema deve possuir um mecanismo centralizado para captura de erros e exceções operacionais, convertendo-os em mensagens amigáveis e estruturadas, evitando a exposição de detalhes internos do código.

RNF-07: Documentação Técnica Interativa

O backend deve expor uma interface viva de documentação baseada no padrão internacional OpenAPI (Swagger), descrevendo todas as rotas operacionais, parâmetros exigidos e estruturas de dados aceitas.

RNF-08: Isolação de Ambientes (Containerização)

A infraestrutura de serviços e o banco de dados devem ser configurados para execução isolada utilizando containers Docker e ferramentas de orquestração,

garantindo estabilidade e paridade total entre as estações de desenvolvimento e o ambiente final de homologação.

RNF-09: Restrição de Acesso de Origens (CORS)

O servidor deve implementar políticas restritivas de compartilhamento de recursos, liberando o processamento de requisições vindas estritamente dos endereços e portas homologados pertencentes à aplicação frontend.

3.3 Requisitos Funcionais — Frontend

RF-10: Interface de Autenticação (Tela de Login)

O sistema deve exibir uma página inicial de login com a identidade visual institucional; validar o preenchimento dos campos obrigatórios no cliente antes de submeter os dados; desabilitar o botão de envio durante o processamento; apresentar alerta em caso de erro de autenticação; e direcionar o usuário para a página de equipes após a confirmação das credenciais.

User Stories:

- Como professor, quero visualizar uma interface clara de login para inserir minhas credenciais e acessar as ferramentas de gestão acadêmica com facilidade.

RF-11: Guarda de Navegação e Proteção de Páginas Restritas

O sistema deve validar de forma contínua o estado de autenticação do usuário antes de renderizar qualquer página restrita do painel e redirecionar automaticamente para a tela de login caso o usuário tente acessar uma URL interna sem estar autenticado.

User Stories:

- Como coordenador, quero que as páginas internas de notas e equipes sejam estritamente protegidas para impedir que terceiros visualizem ou alterem dados dos alunos.

RF-12: Menu de Navegação Estruturado (Sidebar)

O sistema deve disponibilizar um menu de navegação fixo na lateral esquerda com atalhos para todas as áreas; destacar visualmente a opção correspondente à página ativa; permitir o colapsamento do menu para ampliação da área de trabalho; e exibir no rodapé o nome do professor conectado e um botão de saída rápida.

User Stories:

- Como professor, quero contar com um menu lateral fixo para alternar entre a listagem de alunos e o painel de notas com um único clique.

RF-13: Painel de Consulta e Busca de Alunos

O sistema deve apresentar a listagem de todos os alunos cadastrados por meio de cartões visuais organizados alfabeticamente; exibir indicador numérico do total de alunos listados; permitir filtragem em tempo real; e abrir um painel completo de detalhes (modal) ao clicar em um aluno específico.

User Stories:

- Como professor, quero pesquisar alunos digitando apenas as primeiras letras do nome para localizar rapidamente um registro sem rolar páginas infinitas.

RF-14: Formulários de Cadastro e Atualização de Alunos

O sistema deve fornecer uma interface com campos dedicados para inserção de matrícula, nome completo, e-mail acadêmico e número de celular; verificar o formato dos campos de e-mail e celular; carregar os dados existentes em modo de edição; e apresentar um banner temporário de sucesso após a confirmação da gravação.

User Stories:

- Como professor, quero que o formulário me avise caso eu digite um e-mail inválido para evitar o cadastro de dados errados de alunos.

RF-15: Listagem Categorizada de Equipes (Grupos)

O sistema deve organizar e separar visualmente os grupos em seções específicas de acordo com o tipo de MVP (Frontend, Backend ou Mobile); disponibilizar filtros

combinatórios; e permitir a alteração imediata do status da equipe por meio de um seletor rápido no próprio cartão.

User Stories:

- Como professor, quero visualizar os grupos divididos por tecnologia para identificar quais equipes estão desenvolvendo o Backend e quais estão focadas no Mobile.

RF-16: Montagem de Equipes com Atribuição de Alunos e Códigos**Automáticos**

O sistema deve fornecer uma interface de cadastro de grupos coletando nome, tipo de tecnologia, ciclo letivo, projeto associado, link do GitHub e seleção de membros; gerar o código de identificação automaticamente; disponibilizar a lista de alunos via checkboxes; e gerenciar adicionais ou remoções de vínculos durante edições.

User Stories:

- Como professor, quero montar uma equipe selecionando os alunos em uma lista com caixas de seleção, simplificando a composição do grupo de MVP.

RF-17: Painel Estatístico e Gráficos de Desempenho (Dashboard)

O sistema deve exibir cartões visuais resumindo as métricas de volume de projetos, contagem de equipes ativas e andamento das entregas; permitir filtrar todos os gráficos simultaneamente por seletores globais de ano e período; e renderizar gráficos dinâmicos de desempenho.

User Stories:

- Como coordenador de curso, quero analisar gráficos de desempenho das turmas para monitorar as médias gerais obtidas nos projetos de ADS.

RF-18: Exportação de Relatórios Consolidados

O sistema deve fornecer funcionalidade de exportação de dados na tela de indicadores gerenciais, gerando um arquivo .xlsx com colunas organizadas contendo

os dados de grupos, tipo de MVP, ciclo letivo, status, nome dos integrantes e respectivas notas.

User Stories:

- Como professor, quero baixar uma planilha com todas as notas das equipes para realizar o fechamento do diário de classe no final do período letivo.

RF-19: Acompanhamento de Cronograma e Status de Entregas

O sistema deve apresentar a listagem de equipes destinadas à avaliação, diferenciando visualmente por cores as que estão pendentes das já avaliadas; e memorizar os filtros de pesquisa aplicados pelo professor, mantendo o contexto de busca intacto ao navegar entre telas.

User Stories:

- Como professor, quero que o sistema salve meus filtros de busca para que eu não precise reconfigurar a pesquisa toda vez que voltar da tela de notas.

RF-20: Lançamento Dinâmico de Avaliações e Notas Individuais

O sistema deve disponibilizar uma interface de avaliação exibindo os dados consolidados da equipe; conter campos para registro da data da apresentação e observações gerais; listar os integrantes com campo de entrada numérica individual (escala 0 a 6); exibir marcadores de confirmação ao lado dos alunos já avaliados; e atualizar automaticamente o status da equipe para Concluído após a consolidação de todas as notas.

User Stories:

- Como professor, quero dar notas individuais para cada membro do grupo, pois o desempenho na apresentação pode variar de um aluno para outro.

3.4 Requisitos Não Funcionais — Frontend

RNF-10: Arquitetura da Interface e Desempenho

A interface do usuário deve ser implementada como aplicação de página única (SPA) utilizando a biblioteca React e a ferramenta de compilação Vite, garantindo navegação instantânea e tempos mínimos de carregamento de tela.

RNF-11: Identidade Visual e Padronização de Design

O design visual e a estilização dos componentes devem ser construídos com base no framework TailwindCSS. A paleta de cores deve adotar obrigatoriamente o verde institucional do UNIFESO como tom primário, assegurando consistência estética em toda a plataforma.

RNF-12: Componentização e Acessibilidade

O frontend deve ser estruturado utilizando componentes modulares, isolados e reutilizáveis (campos de entrada, botões e alertas padronizados), integrando primitivos de interface acessíveis para garantir estabilidade visual e facilidade em manutenções preventivas.

RNF-13: Gerenciamento Global do Estado de Autenticação

O controle de permissões e a verificação do estado do usuário logado devem ser gerenciados centralizadamente por meio de contextos globais da aplicação. As informações básicas de sessão devem ser mantidas de forma segura no armazenamento do navegador para evitar deslogamentos involuntários durante atualizações de página.

RNF-14: Cliente de Comunicação HTTP Unificado

Todo o consumo de dados vindo dos serviços do servidor deve ser operado por um cliente HTTP padronizado. O sistema deve interceptar as requisições para injetar automaticamente as credenciais de segurança em todas as chamadas destinadas a rotas privadas.

RNF-15: Renovação Transparente de Sessões Expiradas

A camada de comunicação do frontend deve possuir um mecanismo inteligente de interceptação de respostas. Caso o servidor retorne uma negação de acesso por expiração de sessão, o sistema deve solicitar a atualização de credenciais em segundo plano e refazer a operação original de forma transparente.

RNF-16: Responsividade e Adaptação de Telas

O layout completo da aplicação deve ser totalmente responsivo, utilizando grids de distribuição flexíveis que reorganizam as colunas e os elementos de forma fluida de acordo com o tamanho do monitor e a resolução da tela.

RNF-17: Experiência do Usuário e Feedback Operacional

A interface deve fornecer respostas visuais instantâneas para cada operação disparada pelo usuário, incluindo sinalizadores de carregamento (spinners), bloqueio temporário de botões para evitar cliques duplos, banners de confirmação auto-removíveis e avisos em cor de destaque posicionados abaixo de campos com preenchimentos incorretos.

RNF-18: Transições Suaves e Micro-animações

A navegação na plataforma deve contar com animações discretas e transições suaves de tamanho e opacidade ao abrir painéis laterais, cartões de listagem ou janelas flutuantes sobrepostas, proporcionando uma experiência de uso fluida e profissional.

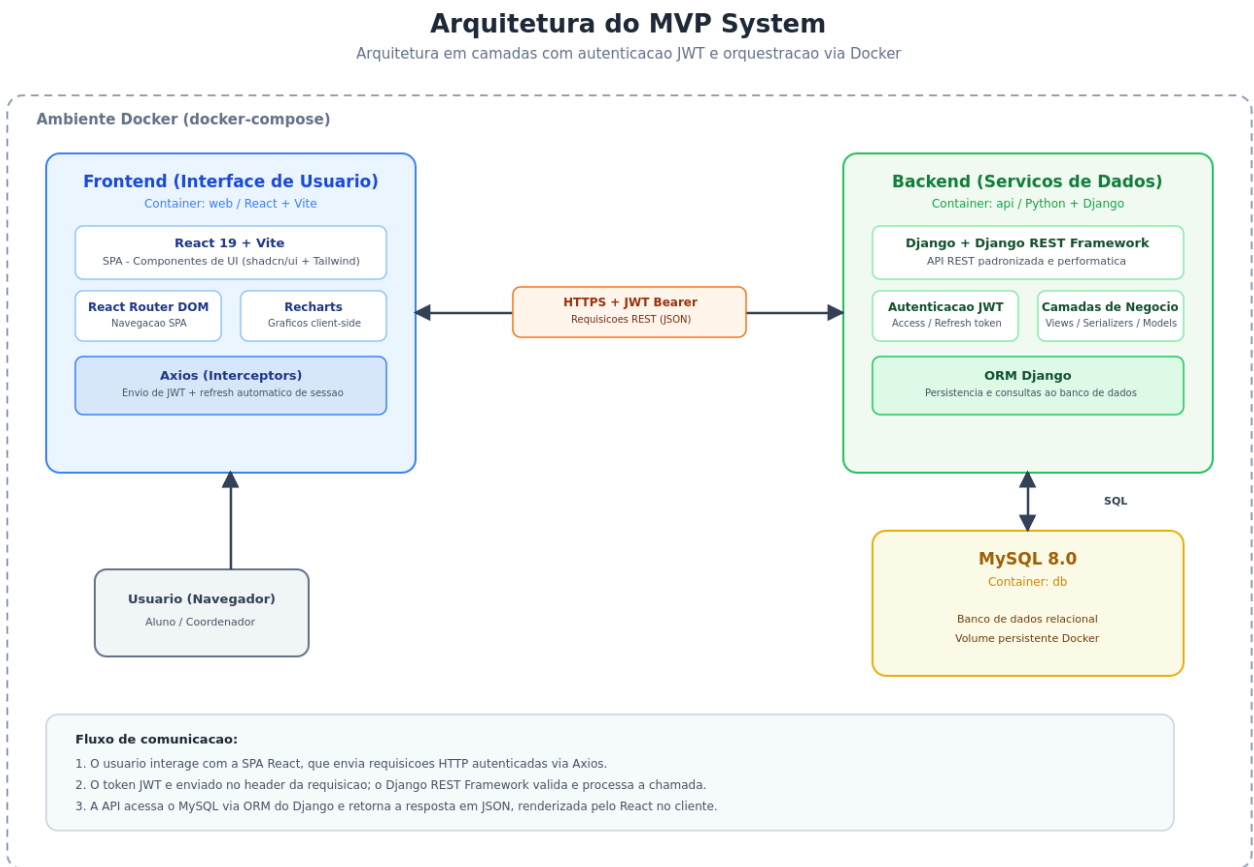
RNF-19: Processamento e Gráficos no Lado do Cliente

A renderização de gráficos estatísticos deve ser realizada no lado do cliente (client-side), permitindo a visualização imediata das informações sem onerar os serviços do servidor de dados.

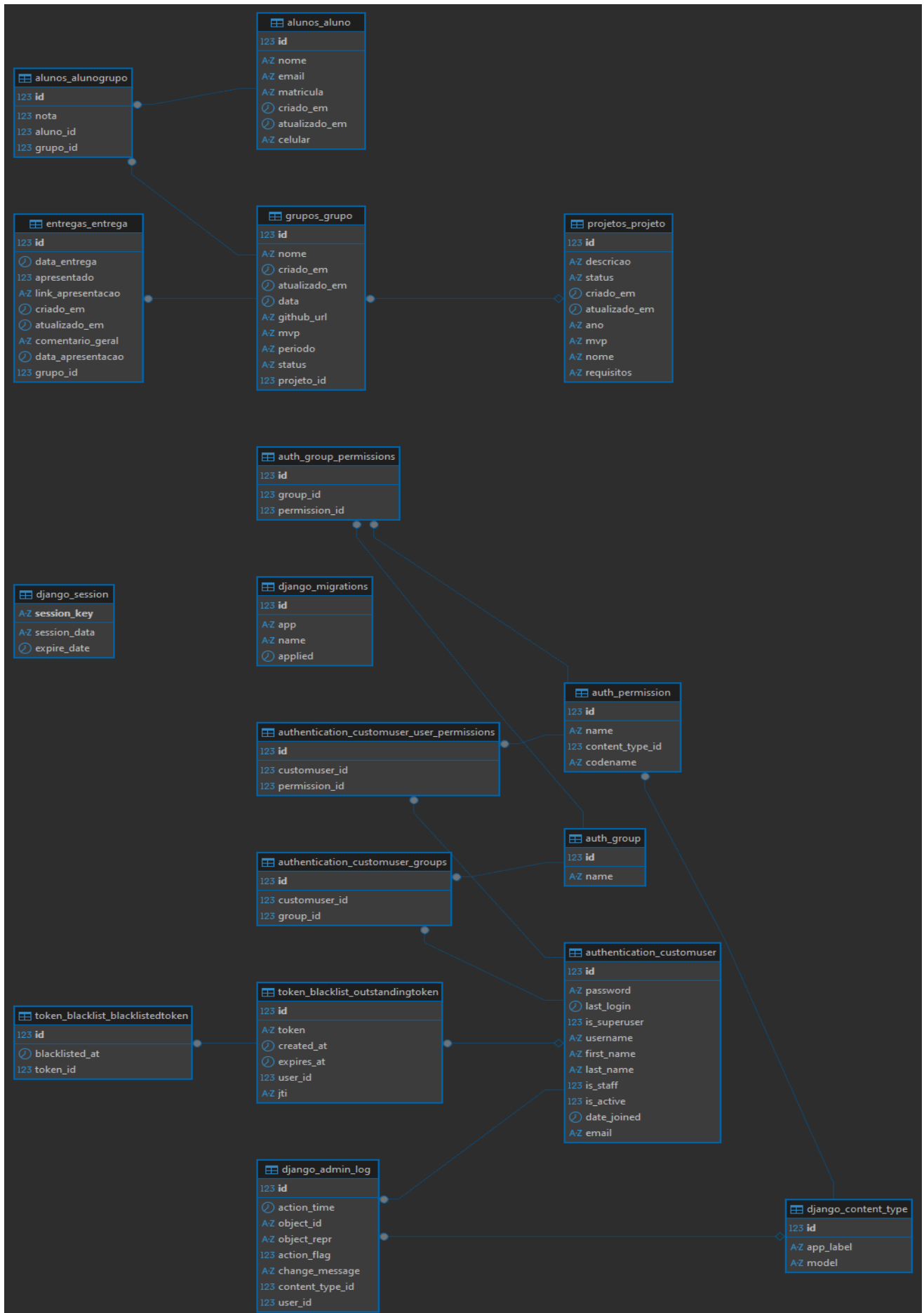
4. Desenvolvimento da Solução

4.1 Arquitetura da Aplicação

O MVP System adota uma arquitetura moderna de camadas, dividida entre uma camada de Serviços de Dados (Backend) e uma Interface de Usuário (Frontend), comunicando-se por meio de protocolos seguros com validação baseada em tokens JWT de alta segurança. A stack tecnológica é composta por Python/Django no servidor, React/Vite no cliente e MySQL como banco de dados relacional, todos orquestrados por containers Docker.



4.2 Modelagem de Dados



4.3 Diagramas de Casos de Uso

Os diagramas a seguir ilustram as interações do ator Professor/Administrador com cada módulo funcional do sistema.

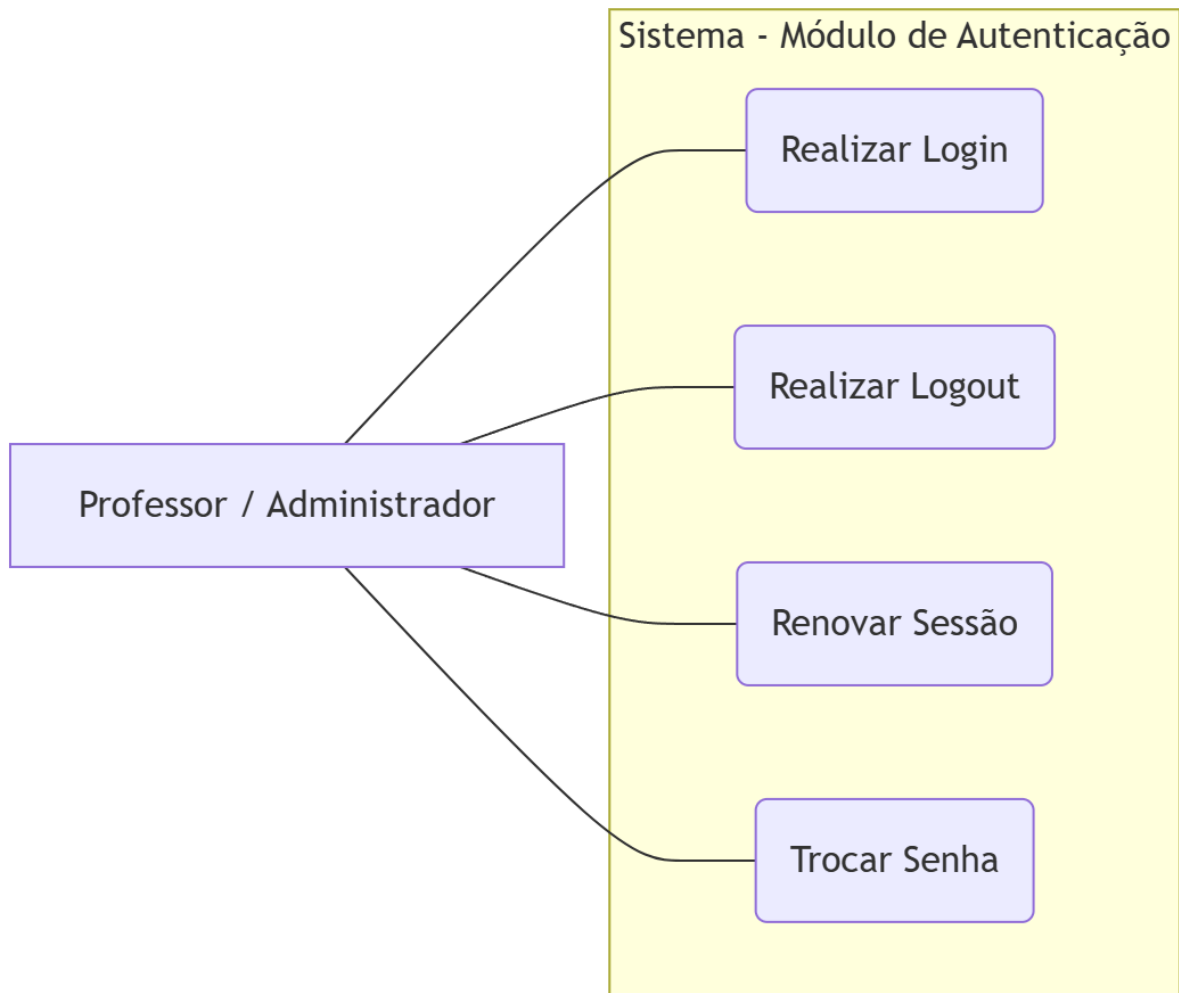


Figura 1 – Diagrama de Casos de Uso: Módulo de Autenticação

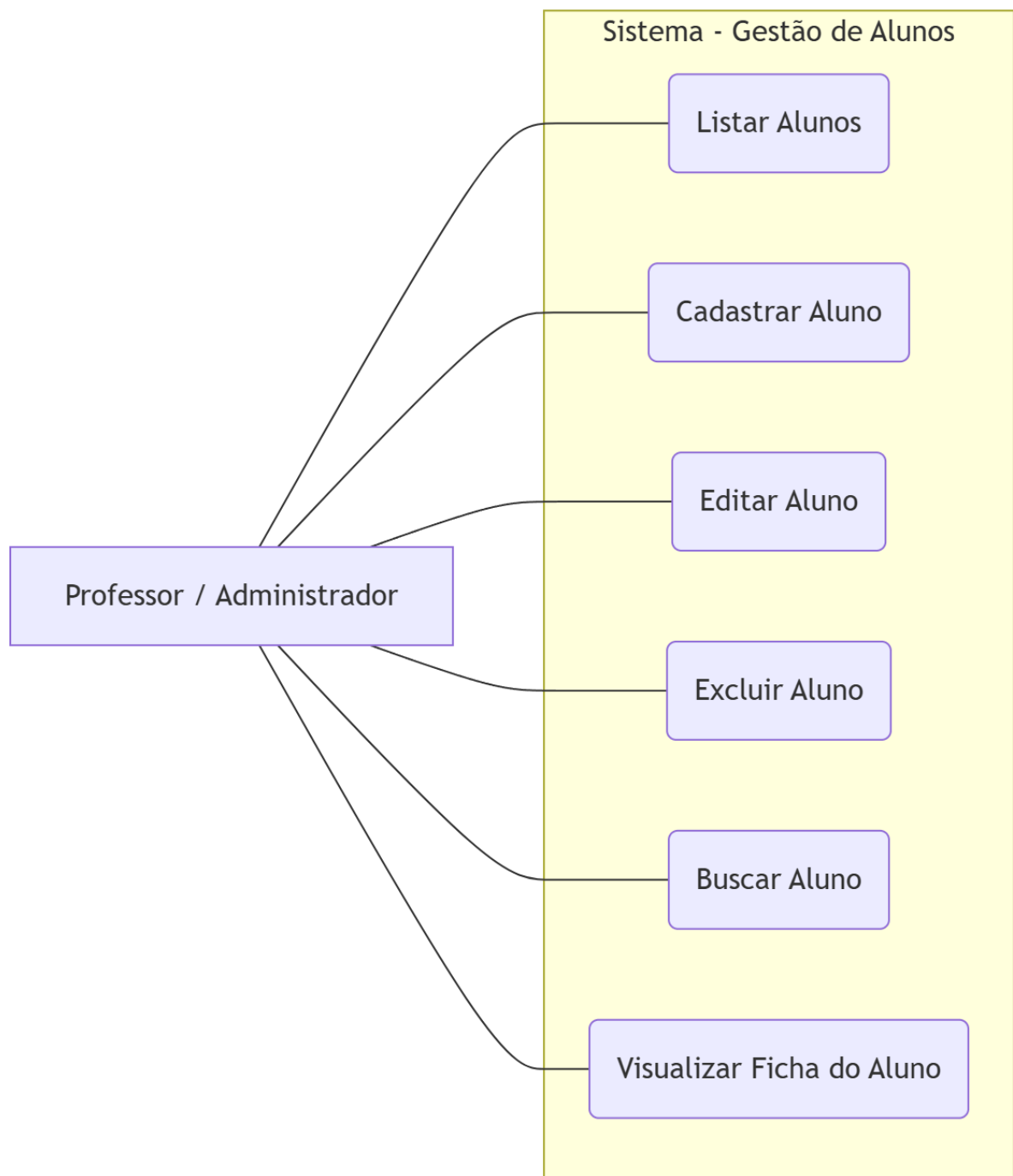


Figura 2 – Diagrama de Casos de Uso: Gestão de Alunos

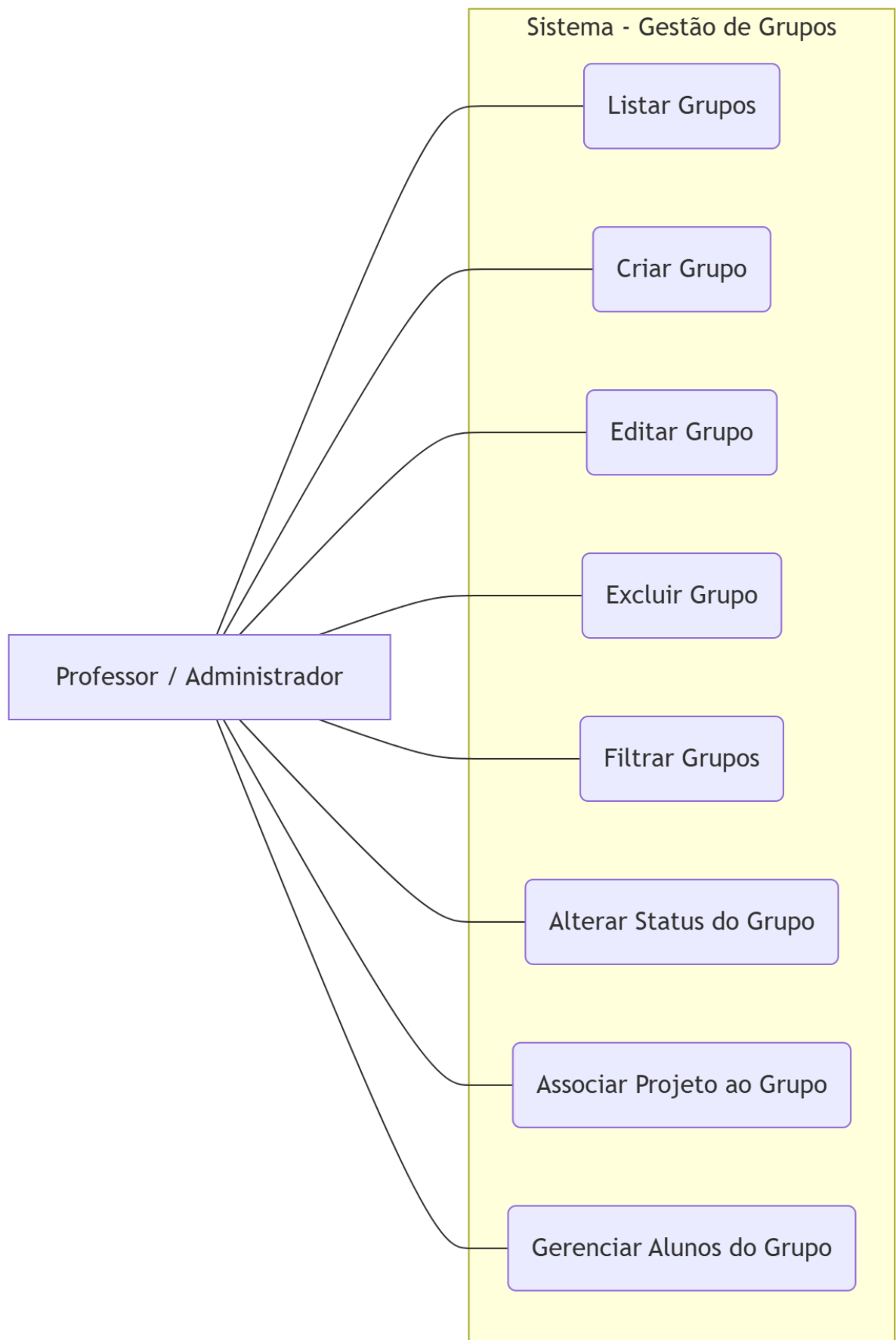


Figura 3 – Diagrama de Casos de Uso: Gestão de Grupos

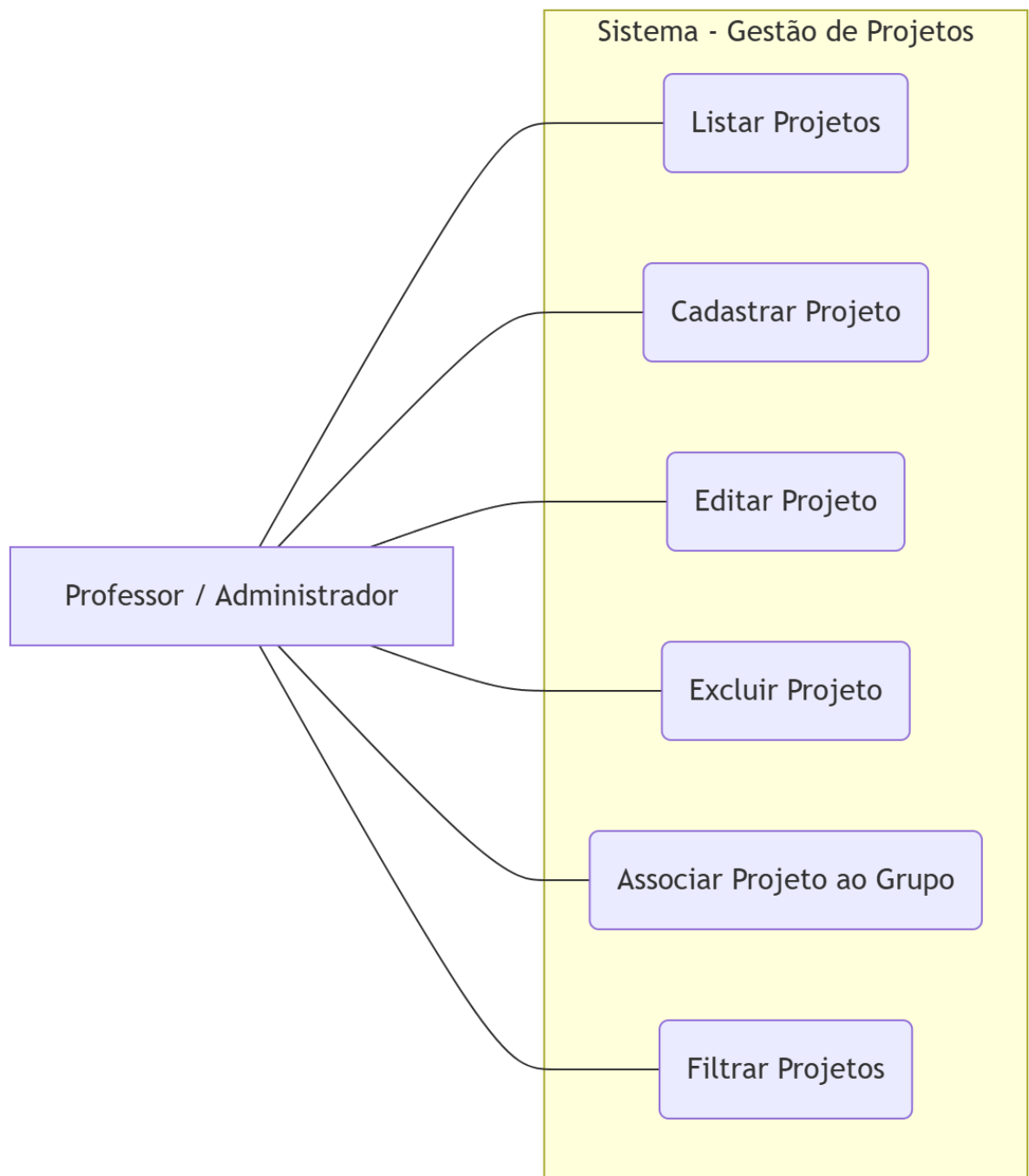


Figura 4 – Diagrama de Casos de Uso: Gestão de Projetos

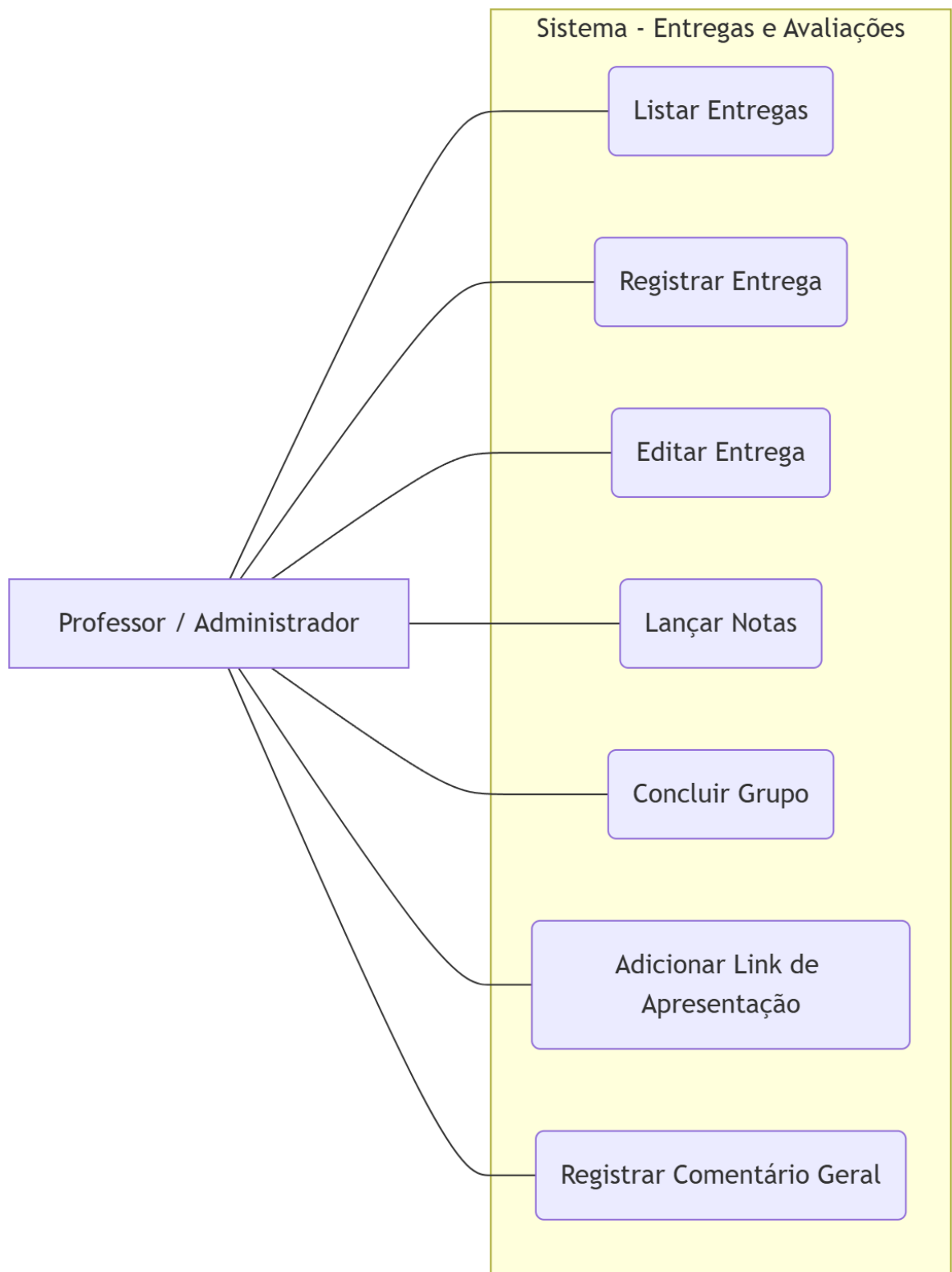


Figura 5 – Diagrama de Casos de Uso: Entregas e Avaliações

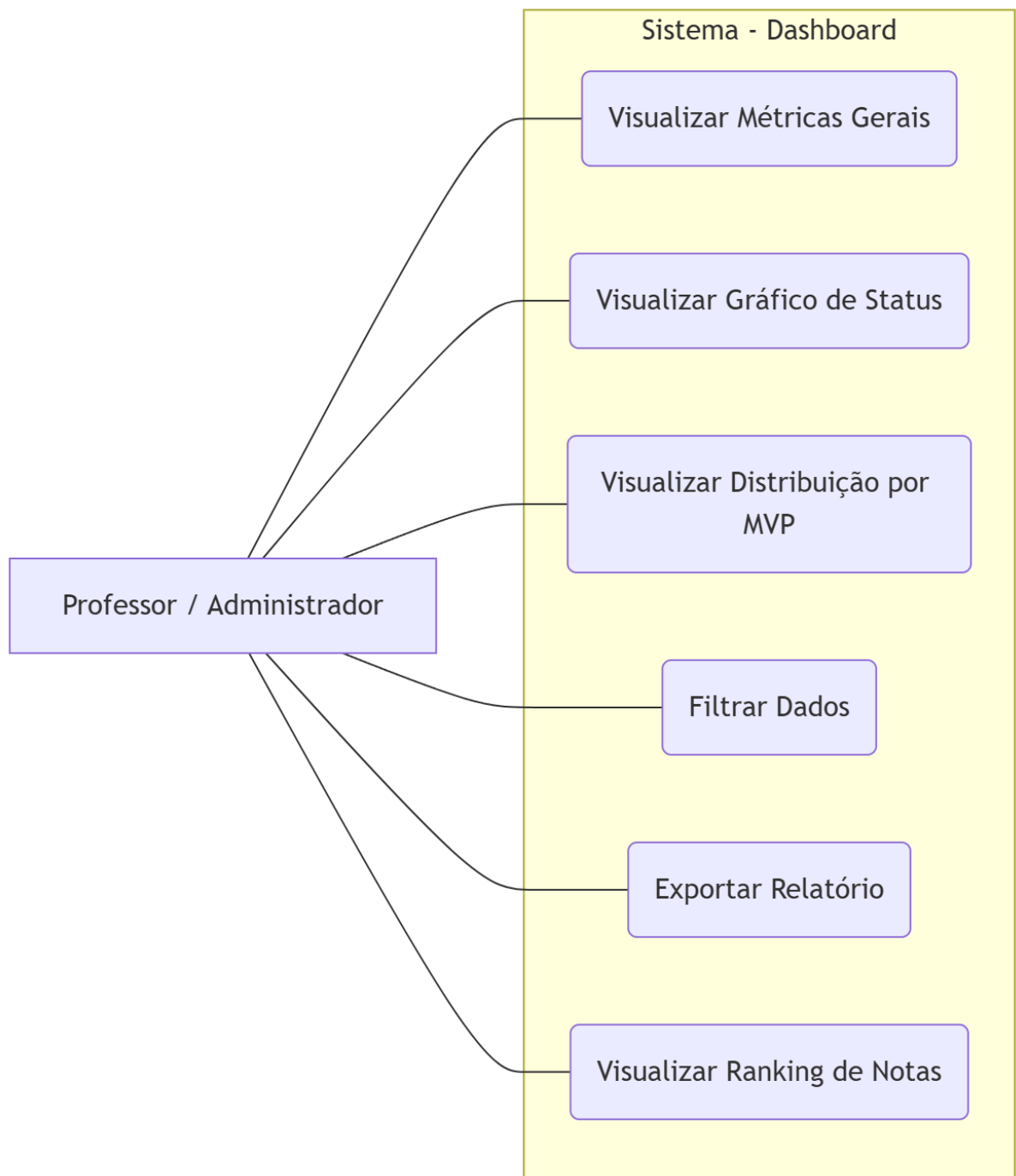


Figura 6 – Diagrama de Casos de Uso: Dashboard

4.4 Diagramas de Sequência

Os diagramas a seguir detalham os principais fluxos operacionais do sistema.

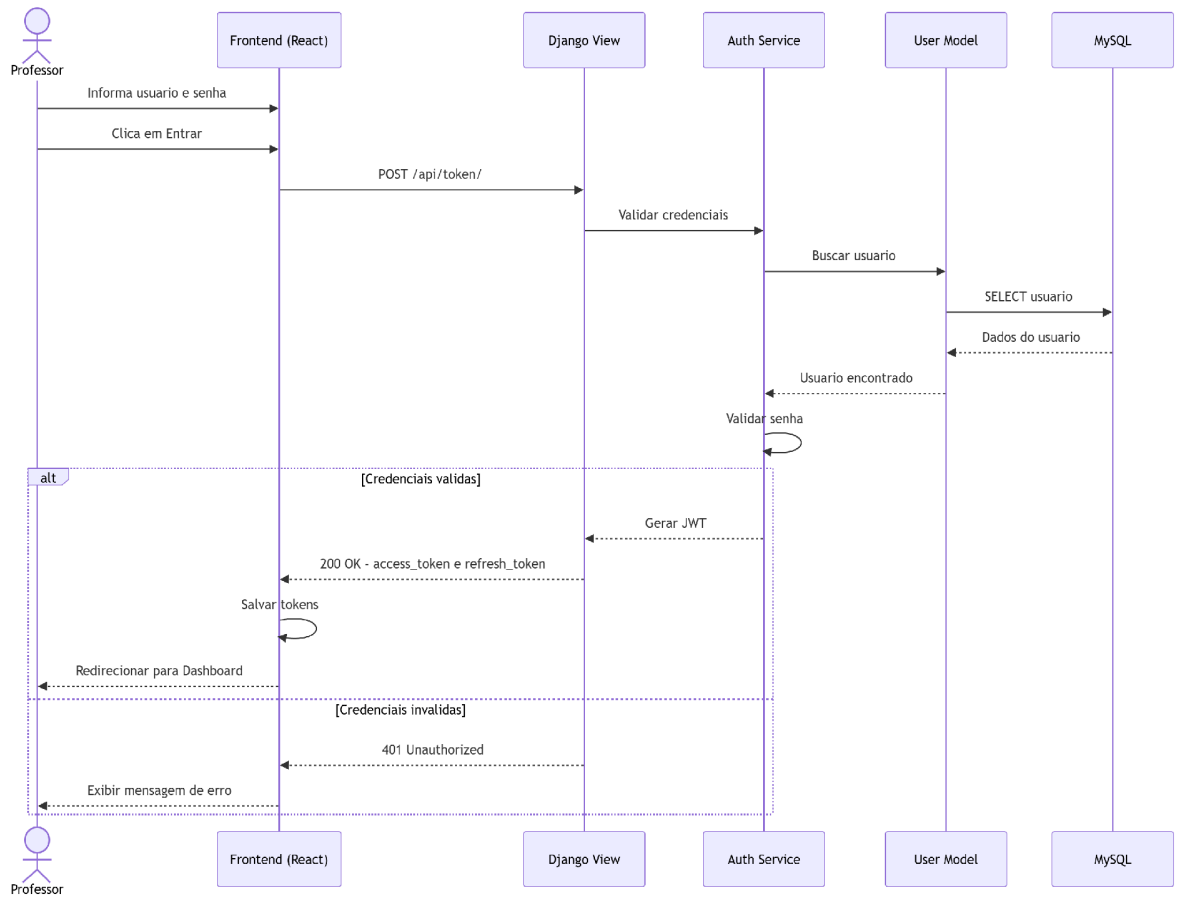


Figura 7 – Diagrama de Sequência: Fluxo de Autenticação JWT

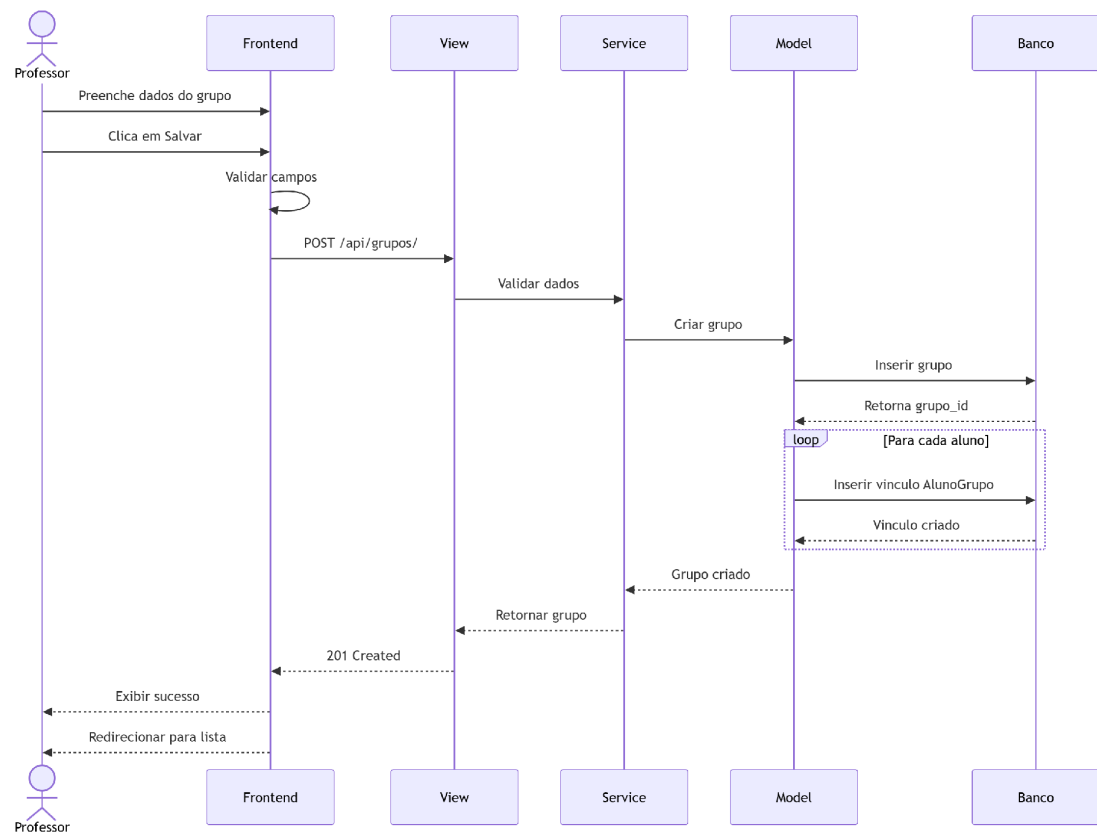


Figura 8 – Diagrama de Sequência: Fluxo de Criação de Grupo

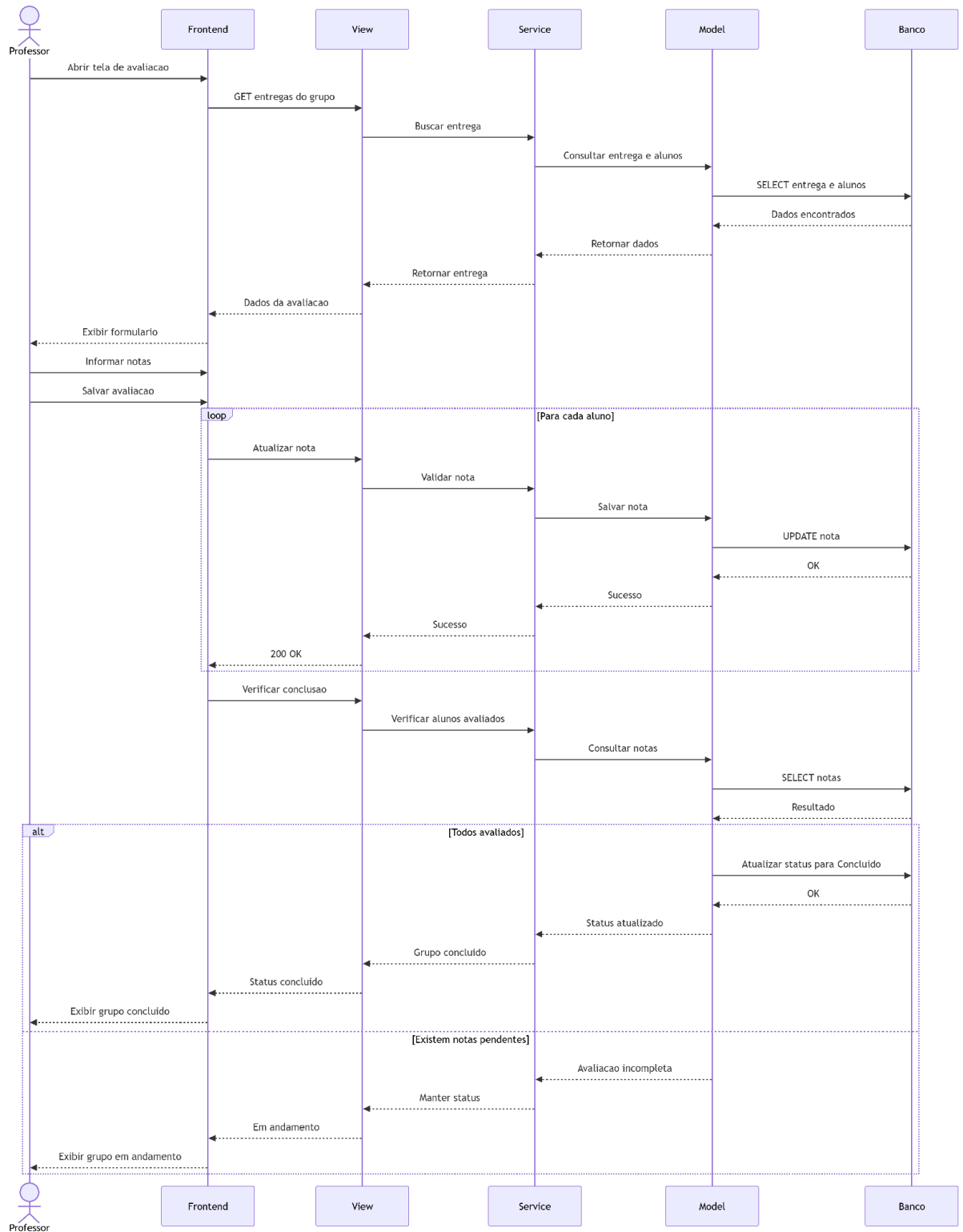


Figura 9 – Diagrama de Sequência: Fluxo de Avaliação de Entregas

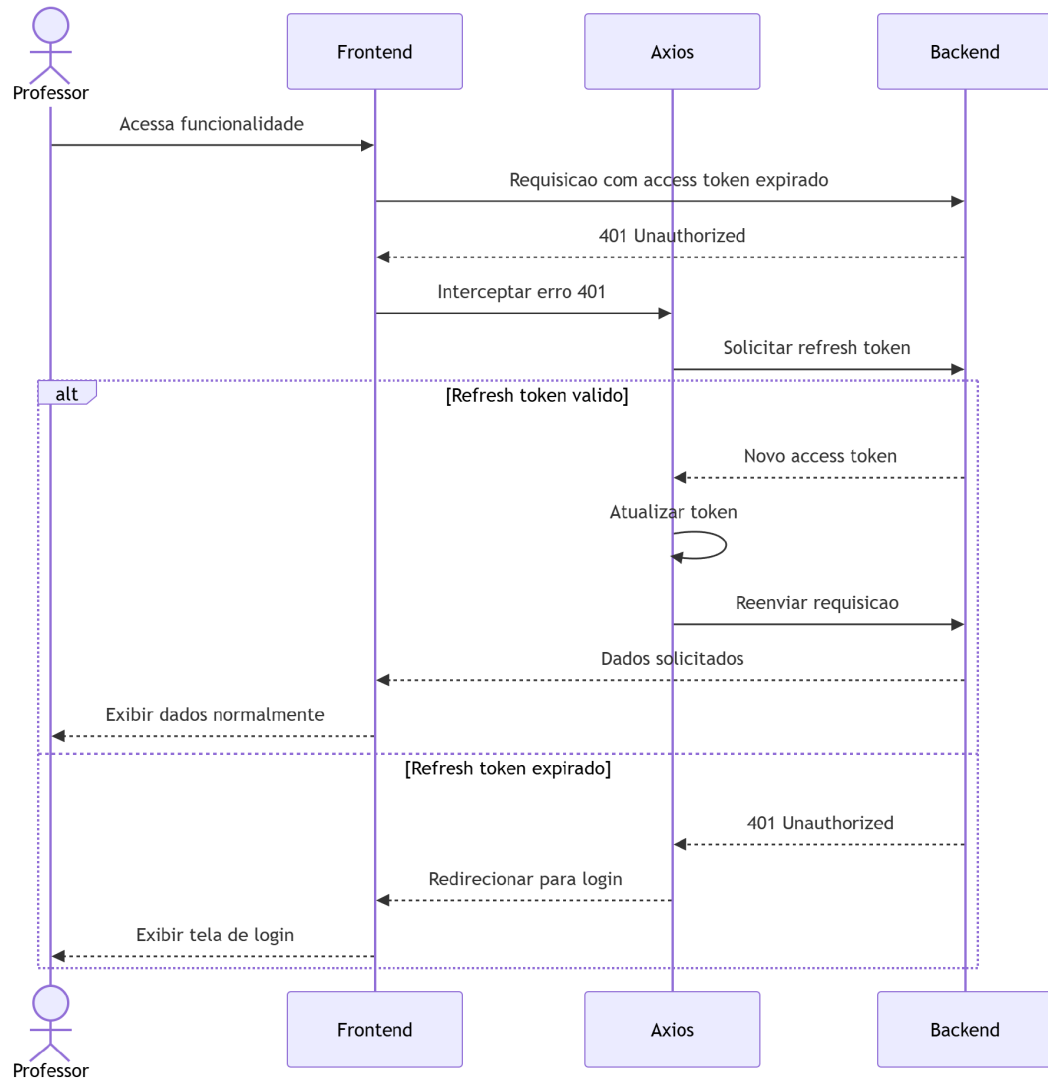


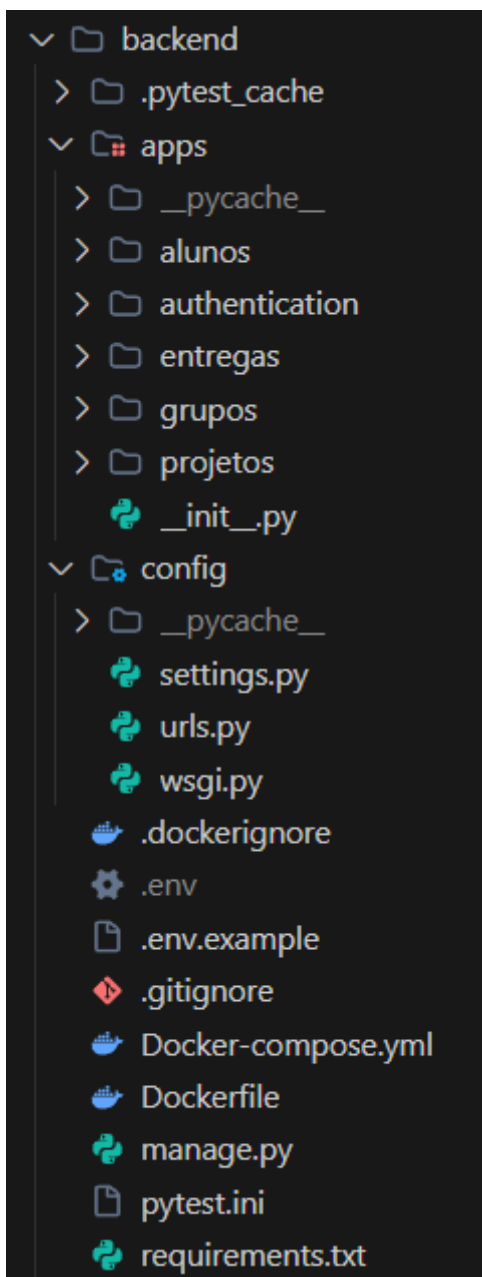
Figura 10 – Diagrama de Sequência: Fluxo de Renovação de Token (Axios Interceptor)

5. Execução do Backend

5.1 Estrutura de Packages

O backend segue uma organização modular por domínio de negócio, garantindo separação de responsabilidades e fácil manutenção. Abaixo estão descritas as estruturas do backend e do frontend:

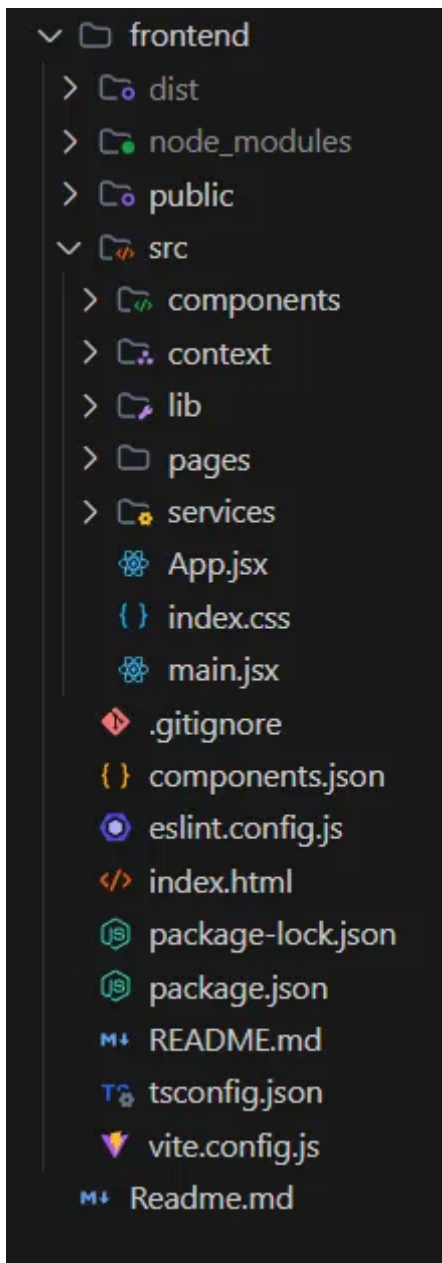
5.1.1 Backend



Arquivo / Diretório	Descrição
<code>backend/</code>	Diretório raiz da aplicação backend.
<code>.pytest_cache/</code>	Cache gerado automaticamente pelo Pytest durante os testes.
<code>apps/</code>	Contém os módulos de negócio da aplicação.
<code>apps/alunos/</code>	Responsável pelo gerenciamento dos alunos.
<code>apps/authentication/</code>	Gerencia autenticação, autorização e usuários.
<code>apps/entregas/</code>	Controla entregas, avaliações e apresentações.
<code>apps/grupos/</code>	Gerencia equipes e grupos de projetos.
<code>apps/projetos/</code>	Responsável pelo cadastro e gerenciamento dos projetos.
<code>apps/__init__.py</code>	Identifica a pasta apps como um pacote Python.
<code>config/</code>	Contém as configurações centrais do projeto Django.
<code>config/__pycache__/</code>	Cache de arquivos Python compilados automaticamente.
<code>config/settings.py</code>	Configurações globais da aplicação Django.
<code>config/urls.py</code>	Define as rotas principais da API.
<code>config/wsgi.py</code>	Ponto de entrada da aplicação para servidores WSGI.
<code>.dockerignore</code>	Define arquivos ignorados durante a criação da imagem Docker.
<code>.env</code>	Armazena variáveis de ambiente e dados sensíveis.
<code>.env.example</code>	Modelo de configuração das variáveis de ambiente.
<code>.gitignore</code>	Define arquivos e diretórios ignorados pelo Git.
<code>docker-compose.yml</code>	Orquestra os containers da aplicação e banco de dados.
<code>Dockerfile</code>	Define como a imagem Docker do backend será construída.
<code>manage.py</code>	Utilitário de gerenciamento e comandos do Django.
<code>pytest.ini</code>	Configurações utilizadas pelo framework de testes Pytest.
<code>requirements.txt</code>	Lista as dependências Python necessárias ao projeto.

5.1.2 Frontend

O frontend adota uma organização modular por responsabilidade, separando componentes visuais, lógica de estado, integração com a API e páginas da aplicação em diretórios específicos. A estrutura é gerenciada pelo Vite e configurada via shadcn/ui.



Arquivo / Diretório	Descrição
<code>frontend/</code>	Diretório raiz da aplicação frontend.
<code>dist/</code>	Arquivos gerados após a compilação da aplicação para produção.

Arquivo / Diretório	Descrição
<code>node_modules/</code>	Dependências instaladas pelo NPM.
<code>public/</code>	Arquivos estáticos acessíveis diretamente pela aplicação.
<code>src/</code>	Contém todo o código-fonte da aplicação React.
<code>src/components/</code>	Componentes reutilizáveis da interface do usuário.
<code>src/context/</code>	Gerenciamento de estado global utilizando Context API.
<code>src/lib/</code>	Configurações e bibliotecas auxiliares da aplicação.
<code>src/pages/</code>	Páginas e telas principais do sistema.
<code>src/services/</code>	Serviços responsáveis pela comunicação com a API.
<code>src/App.jsx</code>	Componente raiz responsável pela estrutura da aplicação.
<code>src/index.css</code>	Estilos globais da aplicação e configurações do Tailwind CSS.
<code>src/main.jsx</code>	Ponto de entrada responsável por renderizar a aplicação React.
<code>.gitignore</code>	Define arquivos e diretórios ignorados pelo Git.
<code>components.json</code>	Arquivo de configuração dos componentes do shadcn/ui.
<code>eslint.config.js</code>	Configuração das regras de análise estática do código JavaScript.
<code>index.html</code>	Página HTML base utilizada pelo Vite para iniciar a aplicação.
<code>package-lock.json</code>	Registro exato das versões das dependências instaladas.
<code>package.json</code>	Manifesto do projeto contendo dependências e scripts de execução.
<code>README.md</code>	Documentação e instruções gerais do projeto frontend.
<code>tsconfig.json</code>	Configuração do compilador TypeScript utilizada por ferramentas do projeto.
<code>vite.config.js</code>	Configuração do ambiente de desenvolvimento e build do Vite.

5.2 Endpoints da API RESTful

Todos os endpoints são protegidos por autenticação JWT, exigindo o envio do access token no cabeçalho Authorization: Bearer <token>. A documentação interativa completa está disponível via Swagger UI no endpoint /api/schema/swagger-ui/.

Autenticação (/api/v1/auth/)

Endpoints responsáveis pelo controle de acesso ao sistema.

Autenticação			Endpoints responsáveis pelo controle de acesso ao sistema. Inclui login, geração e renovação de tokens JWT, logout com invalidação de sessão, cadastro de novos usuários, alteração de senha e consulta do perfil do usuário logado.	
PATCH	/api/v1/auth/change-password/			
POST	/api/v1/auth/logout/			
GET	/api/v1/auth/me/			
POST	/api/v1/auth/register/			
POST	/api/v1/auth/token/			
POST	/api/v1/auth/token/refresh/			
GET	/api/v1/auth/users/			
DELETE	/api/v1/auth/users/{id}/			

Figura 11 – Endpoints de Autenticação (Swagger UI)

Alunos (/api/v1/alunos/)

Endpoints para gerenciamento completo dos alunos.

Alunos			Endpoints para gerenciamento completo dos alunos. Permite cadastrar, listar, buscar, editar, excluir e vincular alunos a grupos de projeto.	
GET	/api/v1/alunos/			
POST	/api/v1/alunos/			
GET	/api/v1/alunos/{id}/			
PUT	/api/v1/alunos/{id}/			
PATCH	/api/v1/alunos/{id}/			
DELETE	/api/v1/alunos/{id}/			
POST	/api/v1/alunos/{id}/desvincular-grupo/			
PATCH	/api/v1/alunos/{id}/lançar-nota/			
POST	/api/v1/alunos/{id}/vincular-grupo/			

Figura 12 – Endpoints de Alunos (Swagger UI)

Grupos (/api/v1/grupos/)

Endpoints para gerenciamento das equipes de projeto.

Grupos Endpoints para gerenciamento das equipes de projeto. Permite criar, listar, editar e excluir grupos, além de consultar os alunos vinculados a cada equipe.			^
GET	/api/v1/grupos/		🔒 ↓
POST	/api/v1/grupos/		🔒 ↓
GET	/api/v1/grupos/{id}/		🔒 ↓
PUT	/api/v1/grupos/{id}/		🔒 ↓
PATCH	/api/v1/grupos/{id}/		🔒 ↓
DELETE	/api/v1/grupos/{id}/	🗑️	🔒 ↓
GET	/api/v1/grupos/{id}/alunos/		🔒 ↓
PATCH	/api/v1/grupos/{id}/vincular-projeto/		🔒 ↓

Figura 13 – Endpoints de Grupos (Swagger UI)

Projetos (/api/v1/projetos/)

Endpoints para gerenciamento dos projetos MVP.

Projetos Endpoints para gerenciamento dos projetos MVP. Permite criar, listar, editar e excluir projetos, com controle de status entre em andamento e concluído.			^
GET	/api/v1/projetos/		🔒 ↓
POST	/api/v1/projetos/		🔒 ↓
GET	/api/v1/projetos/{id}/		🔒 ↓
PUT	/api/v1/projetos/{id}/		🔒 ↓
PATCH	/api/v1/projetos/{id}/		🔒 ↓
DELETE	/api/v1/projetos/{id}/		🔒 ↓
PATCH	/api/v1/projetos/{id}/status/		🔒 ↓

Figura 14 – Endpoints de Projetos (Swagger UI)

Entregas (/api/v1/entregas/)

Endpoints para gerenciamento das entregas e apresentações.

Entregas Endpoints para gerenciamento das entregas e apresentações. Permite registrar entregas, marcar como apresentadas e adicionar links de apresentação por projeto.			^
GET	/api/v1/entregas/	Listar todas as entregas	🔒
POST	/api/v1/entregas/	Criar uma nova entrega	🔒
GET	/api/v1/entregas/{id}/	Obter detalhes de uma entrega	🔒
PUT	/api/v1/entregas/{id}/	Atualizar entrega (completo)	🔒
PATCH	/api/v1/entregas/{id}/	Atualizar entrega (parcial)	🔒
DELETE	/api/v1/entregas/{id}/	Deletar uma entrega	🔒
PATCH	/api/v1/entregas/{id}/apresentar/	Marcar entrega como apresentada	🔒
GET	/api/v1/entregas/grupo/{id}/	Listar entregas de um grupo	🔒

Figura 15 – Endpoints de Entregas (Swagger UI)

5.3 Respostas HTTP

A seguir estão detalhados os códigos de resposta HTTP retornados por cada operação da API:

AuthController

Login — POST /api/v1/auth/token/

- 200: Par de tokens retornado com sucesso (access e refresh).
- 400: Campos username ou password ausentes.
- 401: Credenciais incorretas (usuário não existe ou senha inválida).

Renovar Token — POST /api/v1/auth/token/refresh/

- 200: Novo access token retornado.
- 400: Campo refresh ausente ou token malformado.
- 401: Refresh token expirado ou invalidado.

Registrar Usuário — POST /api/v1/auth/register/

- 201: Usuário criado com sucesso.
- 400: Dados inválidos (username muito curto, username/email já cadastrados, senha sem letras, sem números ou com menos de 8 caracteres).

Logout — POST /api/v1/auth/logout/

- 200: Logout realizado com sucesso.
- 400: Refresh token não informado ou inválido.

- 401: Access token ausente ou inválido.

Alterar Senha — PATCH /api/v1/auth/change-password/

- 200: Senha alterada com sucesso.
- 400: Campos obrigatórios ausentes, senha atual incorreta ou nova senha idêntica à atual.
- 401: Token ausente ou inválido.

AlunoController

Cadastrar Aluno — POST /api/v1/alunos/

- 201: Aluno cadastrado com sucesso.
- 400: Dados inválidos (e-mail ou matrícula já cadastrados, campos obrigatórios ausentes).
- 401: Token ausente ou inválido.

Excluir Aluno — DELETE /api/v1/alunos/{id}/

- 200: Aluno deletado com sucesso.
- 400: Aluno não encontrado ou vinculado a algum grupo.
- 401: Token ausente ou inválido.

Vincular Aluno a Grupo — POST /api/v1/alunos/{id}/vincular-grupo/

- 200: Aluno vinculado ao grupo com sucesso.
- 400: Aluno não encontrado, grupo não encontrado ou aluno já vinculado ao grupo.
- 401: Token ausente ou inválido.

Lançar Nota — PATCH /api/v1/alunos/{id}/lançar-nota/

- 200: Nota lançada com sucesso.
- 400: Aluno não encontrado, grupo não encontrado, aluno não vinculado ao grupo ou nota fora do intervalo permitido (0–6).
- 401: Token ausente ou inválido.

GrupoController

Cadastrar Grupo — POST /api/v1/grupos/

- 201: Grupo cadastrado com sucesso.
- 400: Nome já cadastrado, campos obrigatórios ausentes ou valores inválidos para período, MVP ou status.
- 401: Token ausente ou inválido.

Excluir Grupo — DELETE /api/v1/grupos/{id}/

- 200: Grupo deletado com sucesso.
- 400: Grupo não encontrado, grupo possui alunos vinculados ou projeto ativo.
- 401: Token ausente ou inválido.

Vincular Projeto ao Grupo — PATCH /api/v1/grupos/{id}/vincular-projeto/

- 200: Projeto vinculado ao grupo com sucesso.
- 400: Grupo não encontrado ou projeto não encontrado.
- 401: Token ausente ou inválido.

ProjetoController**Alterar Status do Projeto — PATCH /api/v1/projetos/{id}/status/**

- 200: Status do projeto atualizado com sucesso.
- 400: Campo status ausente ou valor inválido.
- 401: Token ausente ou inválido.

EntregaController**Criar Entrega — POST /api/v1/entregas/**

- 201: Entrega criada com sucesso.
- 400: Grupo não encontrado, campos obrigatórios ausentes, aluno não vinculado ao grupo ou nota inválida.
- 401: Token ausente ou inválido.

Marcar Entrega como Apresentada — PATCH /api/v1/entregas/{id}/apresentar/

- 200: Entrega marcada como apresentada com sucesso.
- 400: Entrega não encontrada ou data de apresentação não informada.

- 401: Token ausente ou inválido.

Excluir Entrega — DELETE /api/v1/entregas/{id}/

- 200: Entrega deletada com sucesso.
- 400: Entrega não encontrada.
- 401: Token ausente ou inválido.

6. Execução da Interface — Frontend

Esta seção destina-se à documentação visual da interface do usuário, com capturas de tela de todas as páginas e fluxos do sistema.

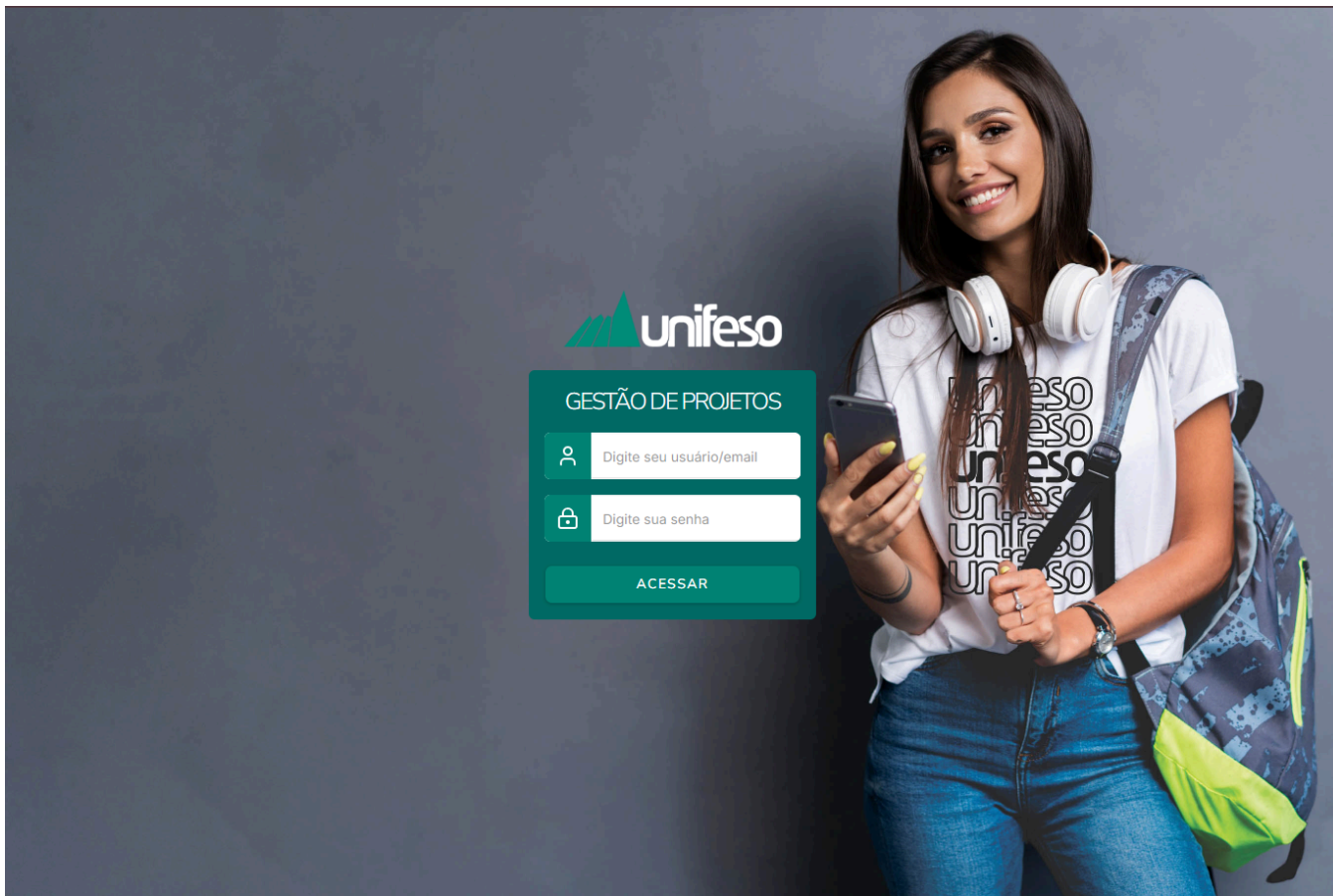


Figura 1 — Tela de Login (Tela_Login)

Observações: Tela de autenticação do sistema. Exibe o logo da UNIFESO e o formulário de login com campos de usuário/e-mail e senha. O botão "ACESSAR" submete as credenciais. A tela possui layout centralizado sobre imagem de fundo com estudante.

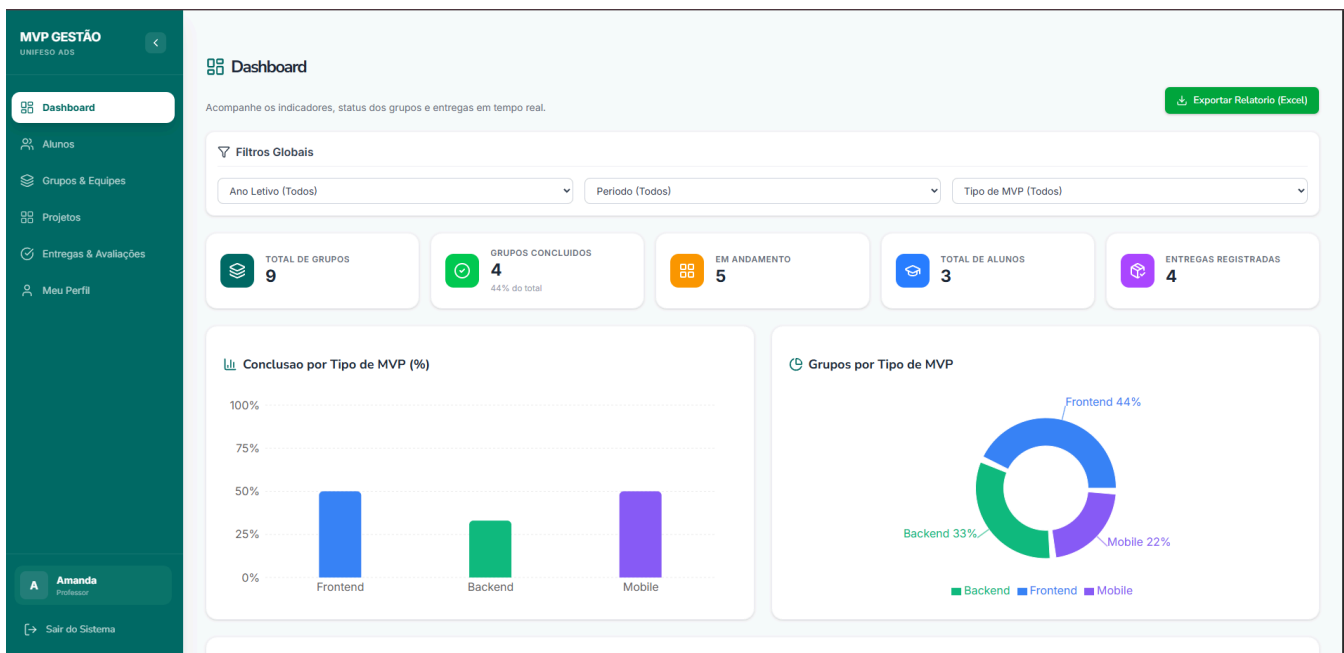


Figura 2 — Tela de Dashboard (Tela_Dashboard)

Observações: Painel principal com indicadores estratégicos do sistema. Exibe filtros globais (ano letivo, período, tipo de MVP), cartões de resumo com total de grupos, grupos concluídos, grupos em andamento, total de alunos e entregas registradas, além de gráficos de conclusão por tipo de MVP e distribuição de grupos por tipo de MVP. Botão "Exportar Relatório (Excel)" no canto superior direito.

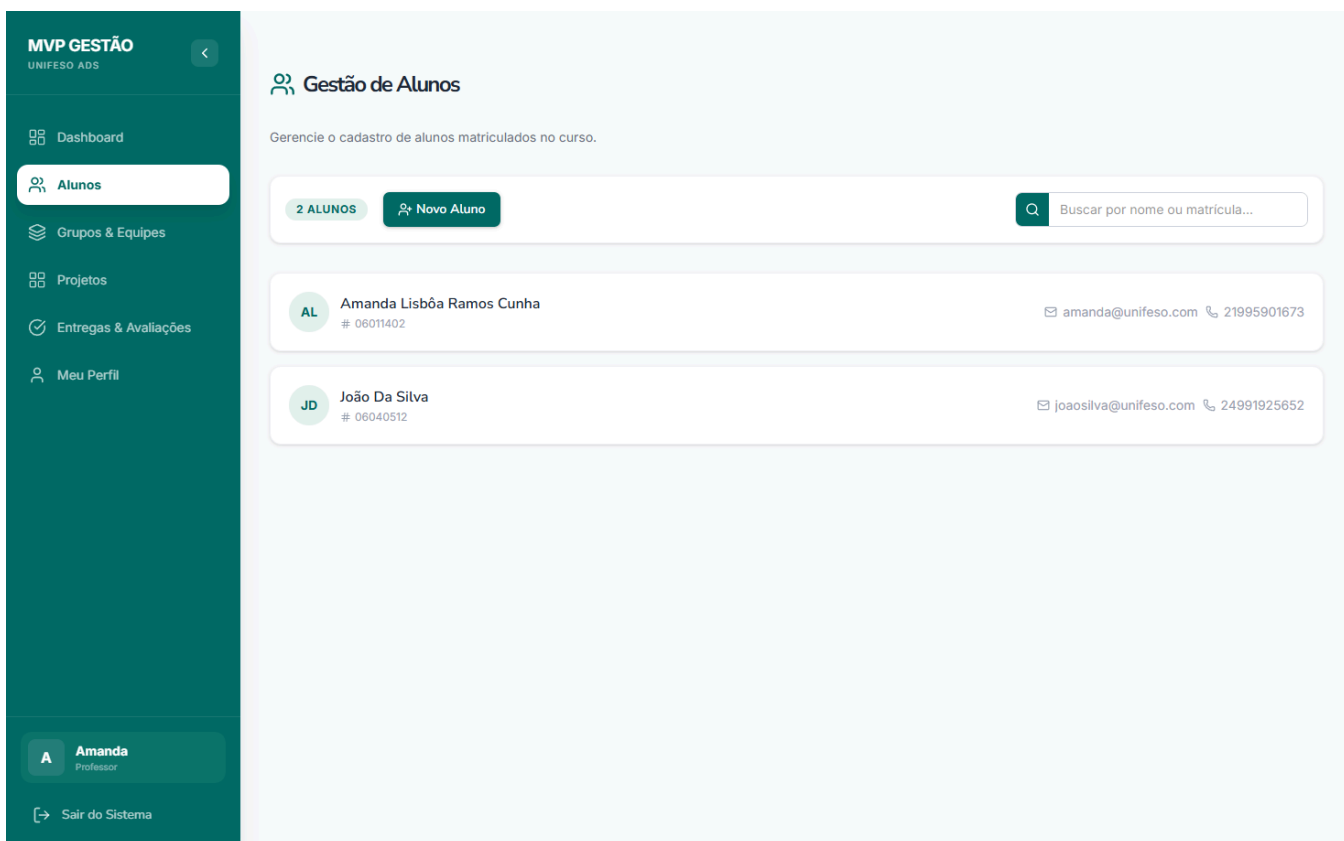


Figura 3 — Tela de Gestão de Alunos (Tela_GestaoAluno)

Observações: Listagem de alunos cadastrados no sistema. Exibe contador de alunos, botão "Novo Aluno" para cadastro e campo de busca por nome ou matrícula. Cada registro apresenta avatar com iniciais, nome completo, matrícula, e-mail acadêmico e telefone. O menu lateral destaca a seção "Alunos" como ativa.

A imagem mostra a interface de usuário para o cadastro de um novo aluno. No topo, há um link de retorno "Voltar para Listagem". O formulário, intitulado "Novo Aluno", contém o seguinte:

- Informações Pessoais** (seção destacada com ícone de identificação):
 - Matrícula ***: Campo com máscara de número, exemplo: 2024001234.
 - Nome Completo ***: Campo com máscara de nome, exemplo: João da Silva.
 - E-mail Acadêmico ***: Campo com máscara de e-mail, exemplo: joao@unifeso.edu.br.
 - Celular (WhatsApp) (Opcional)**: Campo com máscara de telefone, exemplo: (22) 99999-1234.

No rodapé do formulário, há dois botões: "Cancelar" (com ícone de X) e "Cadastrar Aluno" (em verde com ícone de documento).

À esquerda, o menu lateral "MVP GESTÃO UNIFESO ADS" está visível, com a opção "Alunos" selecionada. No rodapé do menu, há o perfil do usuário "Amanda Professor" e o link "Sair do Sistema".

Figura 4 — Tela de Novo Aluno (Tela_NovoAluno)

Observações: Formulário de cadastro de novo estudante. Contém campos obrigatórios de Matrícula e Nome Completo, E-mail Acadêmico, e campo opcional de Celular (WhatsApp). Botões de "Cancelar" e "Cadastrar Aluno" ao final. Link de retorno à listagem no topo da página.

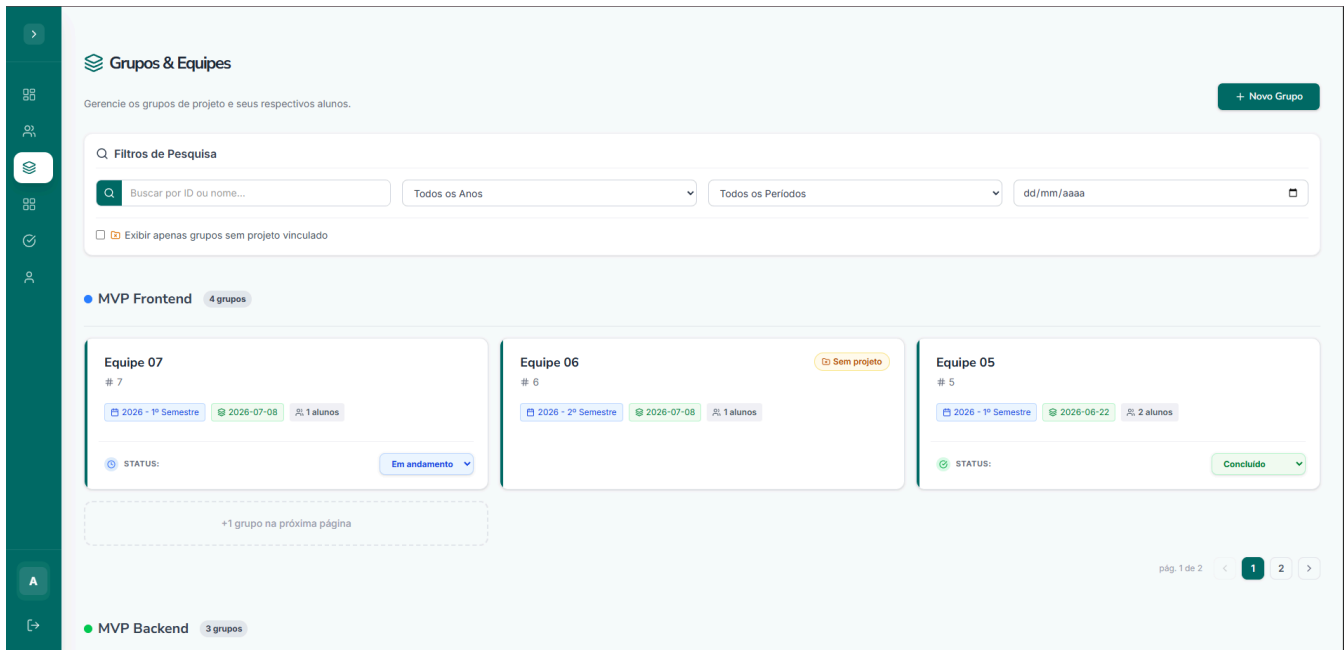


Figura 5 — Tela de Grupos & Equipes (Tela_GruposEquipes)

Observações: Listagem de grupos organizados por tipo de MVP (Frontend, Backend, Mobile). Exibe filtros de pesquisa por ID/nome, ano, período e data, além de opção para exibir apenas grupos sem projeto vinculado. Cada card de equipe traz número identificador, ano letivo, período, data de criação, quantidade de alunos e um seletor de status (Em andamento/Concluído). Botão "Novo Grupo" no canto superior direito.

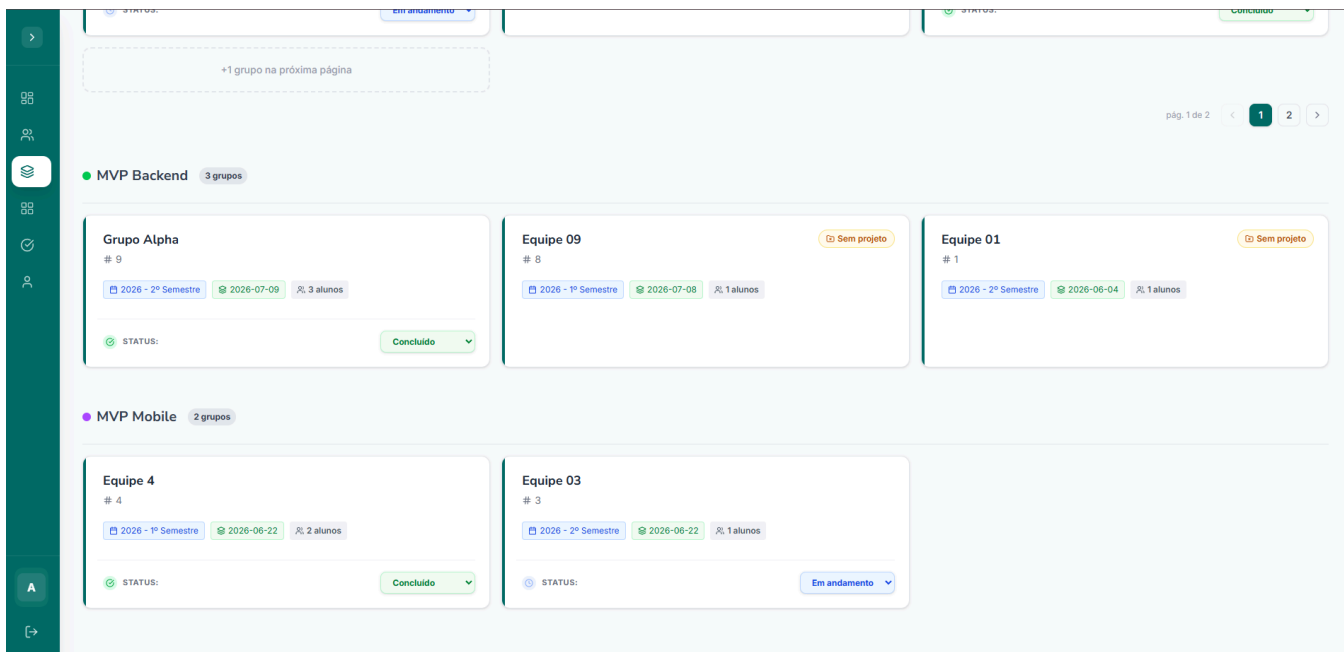


Figura 6 — Tela de Grupos & Equipes – Continuação (Tela_GruposEquipes_2)

Observações: Continuação da listagem de grupos, exibindo as categorias "MVP Backend" e "MVP Mobile". Mesmo padrão de cards com número, ano/período, data de criação, quantidade de alunos e seletor de status, incluindo indicação de grupos "Sem projeto" vinculado.

Observações: Formulário de cadastro de nova equipe, dividido nas seções "Identificação do Grupo" (nome da equipe e tipo de MVP), "Calendário Acadêmico" (data de criação, ano letivo derivado da data e período), "Membros da Equipe" (busca e seleção de alunos por nome ou matrícula) e "Vínculos Externos" (projeto vinculado opcional e link do repositório GitHub). Botões de "Cancelar" e "Cadastrar Equipe" ao final.



Observações: Listagem de projetos em layout de cards. Filtros por nome, tipo de MVP e status. Cada card exibe status (Ativo/Inativo), ano letivo, nome, descrição resumida, tag de tipo (Mobile/Frontend/Backend) e ações de editar, excluir, alternar status e ver detalhes. Botão "Novo Projeto" no canto superior direito.

← Voltar para Projetos

Novo Projeto

Defina os objetivos e requisitos técnicos para os alunos.

Identificação do Projeto

Nome do Projeto *
Ex: Sistema de Gestão Acadêmica

Ano Letivo *
2026

Status *
Ativo

Descrição e Objetivos

Descrição Detalhada *
Descreva o que os alunos deverão desenvolver neste projeto...

Tipo de MVP *

Frontend Backend Mobile

Requisitos / Critérios de Avaliação (Opcional)
Critérios técnicos, ferramentas obrigatórias, etc...

Cancelar Cadastrar Projeto

Figura 9 — Tela de Novo Projeto (Tela_NovoProjeto)

Observações: Formulário de cadastro de projeto com seções "Identificação do Projeto" (nome, ano letivo, status) e "Descrição e Objetivos" (descrição detalhada, tipo de MVP com seleção Frontend/Backend/Mobile, e requisitos/critérios de avaliação opcionais). O menu lateral está recolhido exibindo apenas ícones.

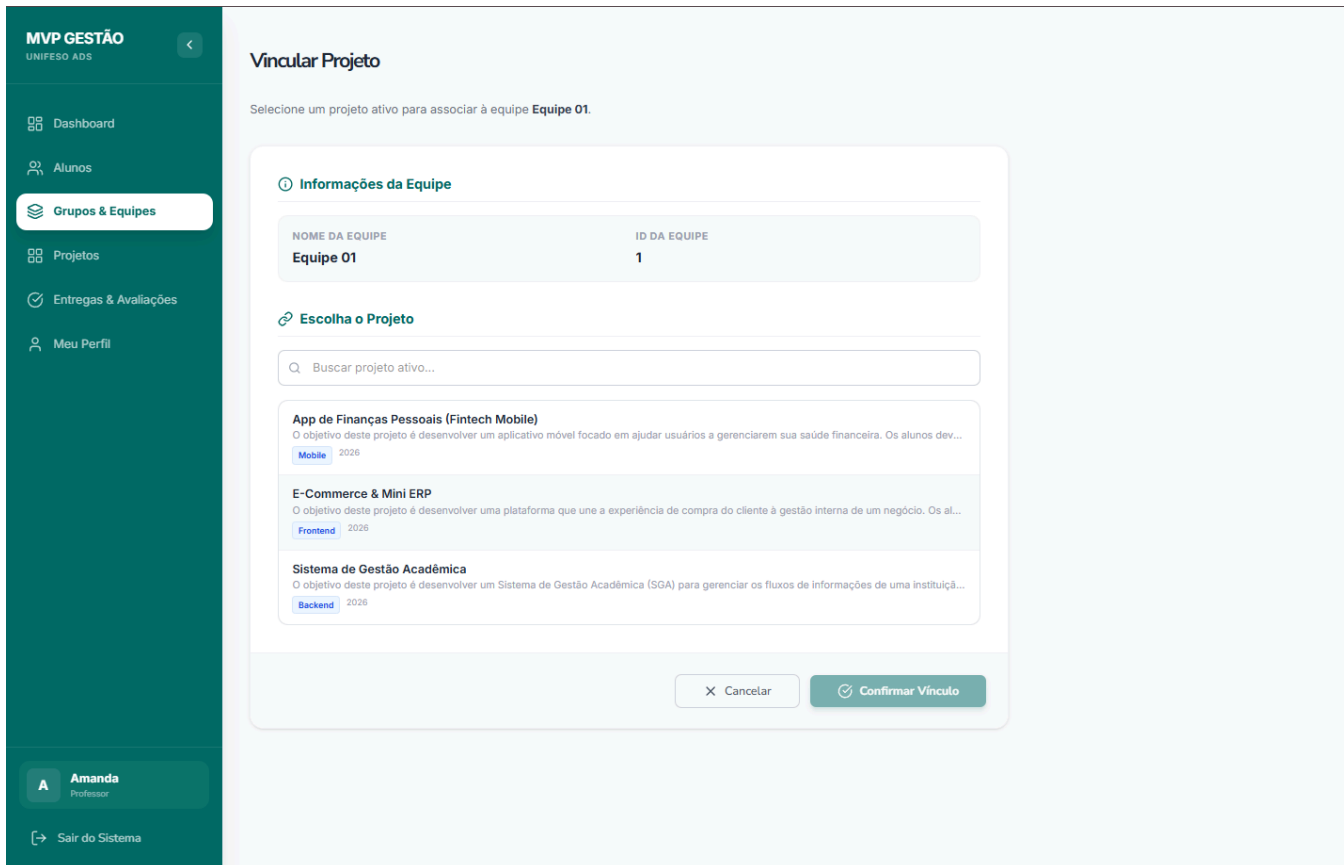


Figura 10 — Tela de Vincular Projeto (Tela_VincularProjeto)

Observações: Tela de vinculação de projeto a uma equipe. Exibe informações da equipe (nome e ID), campo de busca de projeto ativo e lista de projetos disponíveis com descrição e tags de tipo/ano. Botões de "Cancelar" e "Confirmar Vínculo". Destaque para a seção "Grupos & Equipes" no menu.

MVP GESTÃO
UNIFESO ADS

Dashboard
Alunos
Grupos & Equipes
Projetos
Entregas & Avaliações
Meu Perfil

A Amanda
Professor

[→] Sair do Sistema

Registro de Entregas e Apresentações

Avalie as apresentações dos grupos e registre as notas individuais.

Total **5** Concluídos **1** Pendentes **4**

Filtros de Busca ✕ Limpar filtros

Buscar por nome... Tipo de MVP (Todos) Ano (Todos) Período (Todos) Em andamento

Equipe 05 Pendente

5 Frontend 2026 — 1º Semestre App de Finanças Pessoais (Fintech Mobile) 2 alunos Avaliar >

Equipe 03 Pendente

3 Mobile 2026 — 2º Semestre App de Finanças Pessoais (Fintech Mobile) 1 aluno Avaliar >

Equipe 02 Pendente

2 Frontend 2026 — 2º Semestre Sistema de Gestão Acadêmica 1 aluno Avaliar >

Equipe 01 Pendente

1 Backend 2026 — 2º Semestre 1 aluno Avaliar >

Figura 11 — Tela de Entregas e Avaliações com Filtro (Tela_EntregasFiltro)

Observações: Tela de Registro de Entregas e Apresentações com resultado filtrado. Exibe contadores de Total, Concluídos e Pendentes. Filtros de busca por nome, tipo de MVP, ano, período e status. Os grupos listados mostram status "Pendente" com botão "Avaliar" e tags de identificação (número, tipo, período, projeto, alunos).

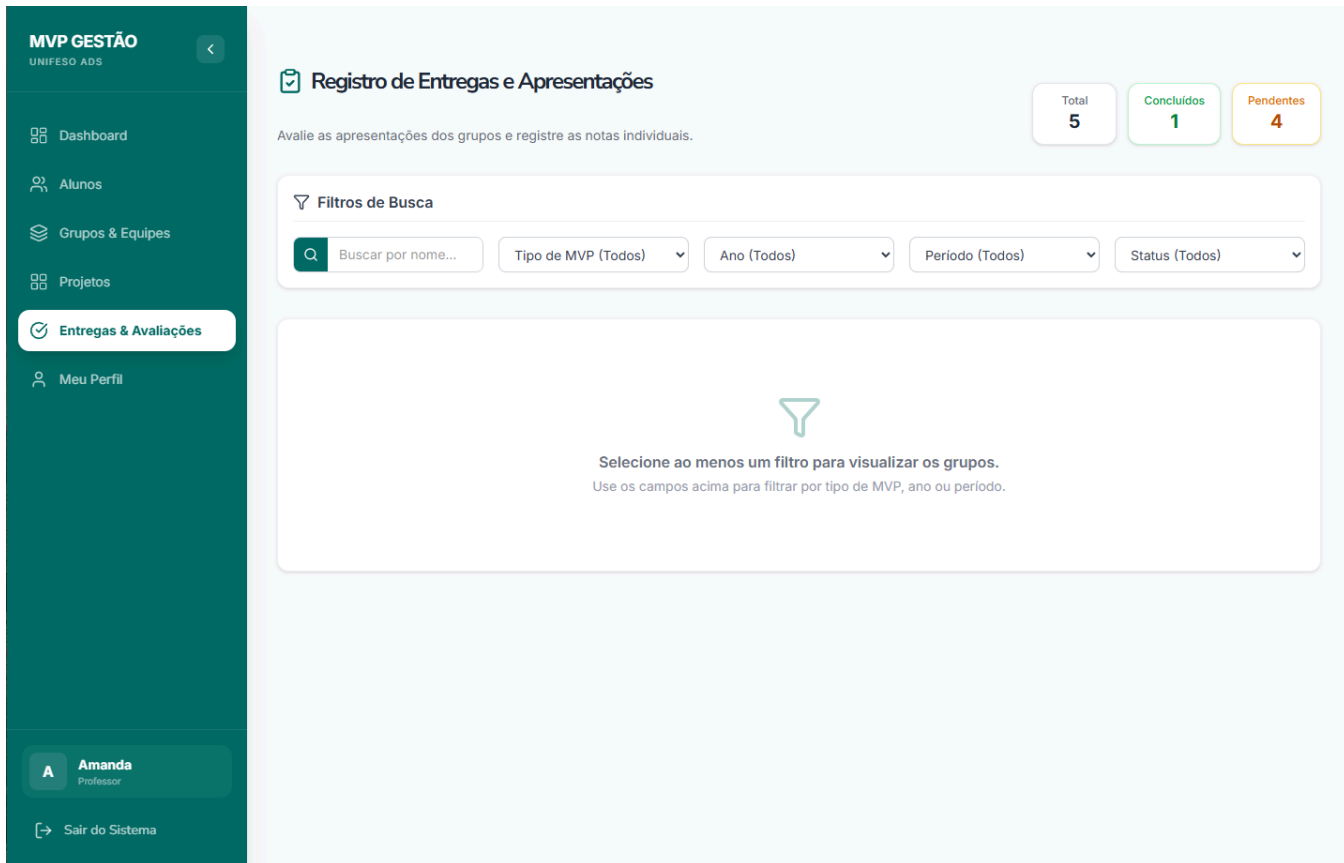


Figura 12 — Tela de Entregas Vazia (Tela_EntregaVazia)

Observações: Estado inicial da tela de Entregas e Avaliações sem filtro aplicado. Exibe mensagem orientativa "Selecione ao menos um filtro para visualizar os grupos" com ícone de funil, instruindo o usuário a usar os campos de filtro disponíveis antes de exibir os resultados.

MVP GESTÃO
UNIFESO ADS

Dashboard
Alunos
Grupos & Equipes
Projetos
Entregas & Avaliações
Meu Perfil

Avaliação do Grupo

Preencha a data de apresentação, comentário geral e as notas individuais.

Dados do Grupo

Equipe 05

5 • Frontend 2026 — 1º Semestre

App de Finanças Pessoais (Fintech Mobile)

Data e Observações

Data de Apresentação

dd/mm/aaaa

Comentário Geral

Observações sobre a apresentação do grupo como um todo...

Avaliação Individual dos Alunos

Nota de 0 a 6 (MVP = 60% da nota)

A **Amanda Lisboa Ramos Cunha**
06011402 Nota: — / 6

J **João Da Silva**
06040512 Nota: — / 6

A **Amanda**
Professor

Sair do Sistema

Cancelar e Voltar Salvar Avaliação

Figura 13 — Tela de Avaliação do Grupo (Tela_AvaliarGrupo)

Observações: Formulário de avaliação individual do grupo. Exibe dados da equipe (nome, número, tipo de MVP, ano/semestre e projeto vinculado), campo de data de apresentação, comentário geral, e lista de alunos com campo de nota individual de 0 a 6 (MVP = 60% da nota). Botões de "Cancelar e Voltar" e "Salvar Avaliação".

MVP GESTÃO
UNIFESO ADS

Dashboard
Alunos
Grupos & Equipes
Projetos
Entregas & Avaliações
Meu Perfil

Perfil e Administração

Gerencie seus dados e os professores com acesso ao sistema.

Meu Perfil Professores

Meus Dados

Informações do seu perfil atual

NOME DE USUÁRIO

Amanda

E-MAIL

teste@teste.com

Sair da conta

Alterar Senha

Atualize sua senha de acesso

SENHA ATUAL *

NOVA SENHA *

Mínimo 8 caracteres

CONFIRMAR NOVA SENHA *

Repita a nova senha

Alterar Senha

A **Amanda**
Professor

Sair do Sistema

Figura 14 — Tela de Meu Perfil (Tela_MeuPerfil)

Observações: Tela de "Perfil e Administração" na aba "Meu Perfil". Exibe seção "Meus Dados" com nome de usuário e e-mail cadastrados, além de opção "Sair da Conta". Seção "Alterar Senha" com campos de senha atual, nova senha e confirmação de nova senha, cada um com opção de exibir/ocultar caracteres.

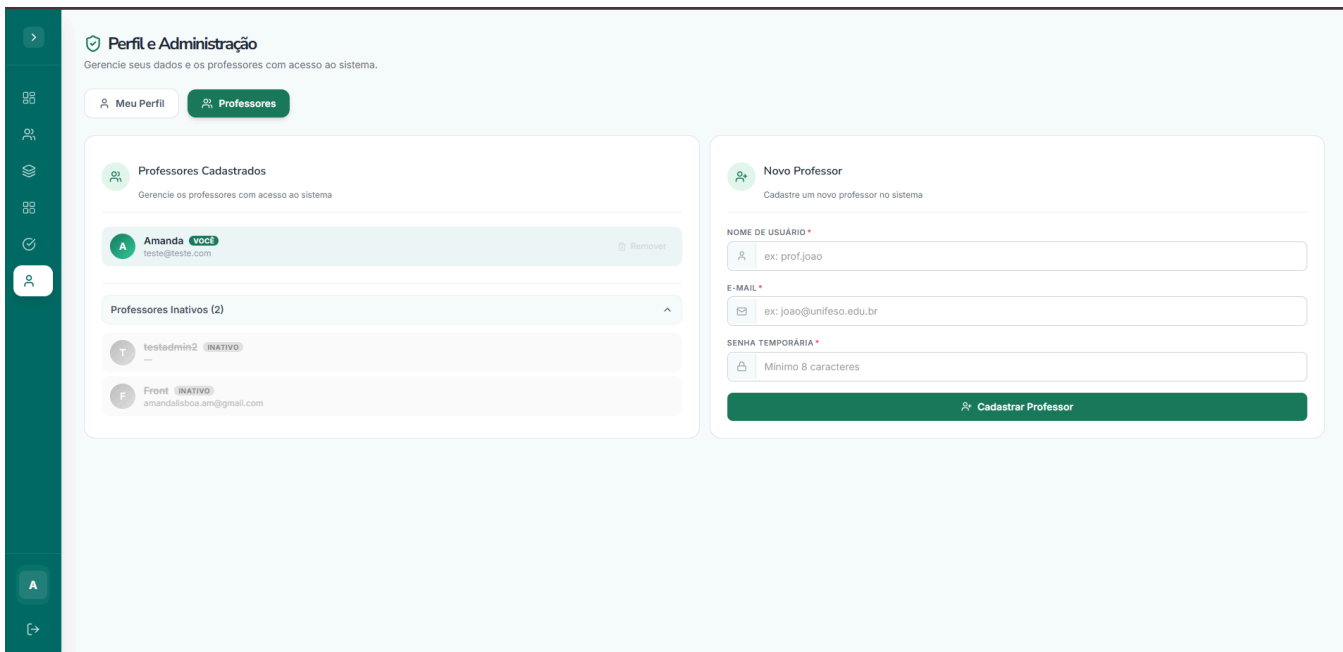


Figura 15 — Tela de Administração de Professores (Tela_Professores)

Observações: Tela de "Perfil e Administração" na aba "Professores". Exibe lista de professores cadastrados com destaque para o usuário atual ("Você") e seção de "Professores Inativos" recolhível. Formulário "Novo Professor" ao lado, com campos de nome de usuário, e-mail e senha temporária, e botão "Cadastrar Professor".

7. Tecnologias Utilizadas

7.1 Backend

O backend é uma API REST desenvolvida integralmente em Python com o framework Django. Todas as versões foram identificadas no arquivo backend/requirements.txt e no Dockerfile.

Tecnologia	Versão	Finalidade no Projeto
Python 3.11	python:3.11-slim (Dockerfile)	Linguagem de programação base de todo o backend. Responsável pela lógica de negócio, configuração e execução do servidor.
Django	>=4.2, <5.0	Framework web de alto nível que provê a estrutura MVC/MVT do projeto: roteamento de URLs, ORM, sistema de migrações, painel administrativo e gerenciamento de usuários.
Django REST Framework	>=3.14.0	Extensão especializada na construção de APIs RESTful. Responsável pela serialização de dados, sistema de permissões e renderização de respostas JSON.
djangorestframework-simplejwt	>=5.3.0	Biblioteca de autenticação baseada em JSON Web Token. Implementa geração e validação de tokens de acesso (15 min) e atualização (7 dias), com rotação e blacklist de sessões.
django-cors-headers	>=4.3.0	Middleware de configuração de políticas CORS, permitindo que o frontend (http://localhost:5173) consuma a API sem bloqueios do navegador.
mysqlclient	>=2.2.0	Driver de conexão nativo entre o Django e o banco de dados MySQL. Traduz operações do ORM em consultas SQL.
python-dotenv	>=1.0.0	Carregamento de variáveis de ambiente a partir do arquivo .env, evitando a exposição de credenciais sensíveis no código-fonte.

7.2 Frontend

O frontend é uma aplicação de página única (SPA) desenvolvida em React. Todas as versões foram extraídas do arquivo frontend/package.json.

Tecnologia	Versão	Finalidade no Projeto
React	^19.2.4	Biblioteca JavaScript principal para construção da interface com base em componentes reutilizáveis e reativos.
Vite	^8.0.4	Ferramenta de build e servidor de desenvolvimento de alto desempenho, com uso de módulos ES nativos. Configurado com os plugins @vitejs/plugin-react e @tailwindcss/vite.
Tailwind CSS	^4.2.2	Framework CSS utilitário integrado ao Vite. Responsável pela estilização da interface por meio de classes utilitárias, com suporte a variáveis CSS.
shadcn/ui	^4.3.0	Sistema de componentes configurável baseado em Radix UI e Tailwind CSS. Provê componentes acessíveis e altamente customizáveis (botões, modais, formulários).
Radix UI	^1.4.3	Biblioteca de primitivas de interface (headless) utilizada como base pelo shadcn. Garante acessibilidade ARIA, comportamento de teclado e semântica HTML corretos.
React Router DOM	^7.14.1	Biblioteca de roteamento client-side para React. Gerencia a navegação entre páginas da SPA sem recarregamento do servidor.
Axios	^1.16.0	Cliente HTTP baseado em Promises. Responsável por todas as requisições assíncronas, incluindo envio de tokens JWT e interceptação para renovação de sessão.
Recharts	^3.8.0	Biblioteca de gráficos responsivos para React, construída sobre SVG. Utilizada para renderização de gráficos no dashboard.

Lucide React	^1.8.0	Biblioteca de ícones SVG minimalistas, integrada como iconLibrary padrão na configuração do shadcn.
--------------	--------	---

7.3 Banco de Dados

Item	Detalhe
Sistema Gerenciador	MySQL
Versão	8.0 (imagem Docker mysql:8.0 declarada no docker-compose.yml)
Tipo	Banco de dados relacional

7.4 Infraestrutura e DevOps

A infraestrutura de desenvolvimento é baseada em containers Docker, garantindo isolamento de ambiente e paridade entre as estações de desenvolvimento e o ambiente de homologação.

Ferramenta	Versão	Finalidade
Docker	20.10.x ou superior	Containerização do ambiente de backend e banco de dados. Define a imagem do serviço web com Python 3.11 Slim.
Docker Compose	3.9	Orquestração multi-container: gerencia simultaneamente os serviços db (MySQL) e web (Django API), incluindo dependência de inicialização com healthcheck e mapeamento de volumes e portas.